模式识别
——模式识别与机器学习

刘文黎

liu_wenli@hust.edu.cn

土木与水利工程学院

第9章 深度学习与大模型

本章主要内容

12.1 引言
12.2 人工神经网络回顾
12.3 卷积神经网络（CNN）
12.4 循环神经网络（RNN）
12.5 长短时记忆模型（LSTM）
12.6 自编码器、限制性玻尔兹曼机与深度信念网络
12.7 生成模型
12.8 综合应用举例
12.9 深度学习算法中的部分常用技巧
12.10 生成式人工智能
12.11 人工智能范式的变化
12.12 ChatGPT
12.13 讨论

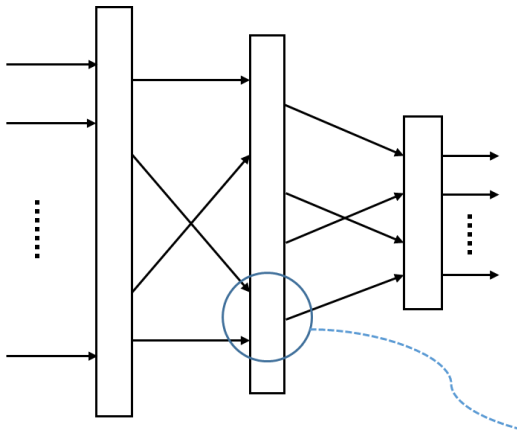
12.1 引言

- 深度学习
  - 1986年，Rina Dechter提出这个词，以搜索和推理为核心
  - 2000年，I. Aizenberg 引入到神经网络研究
  - 具有多层结构神经网络模型的机器学习方法
  - 是一大类机器学习方法的总称

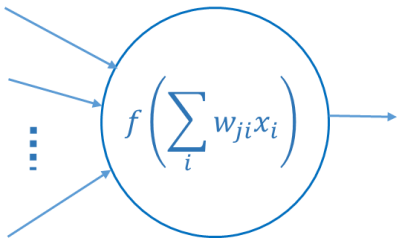
- 深度学习也与神经网络与贝叶斯学习等结合起来，并不限于狭义的神经网络的概念

12.2 人工神经网络回顾

- 多层感知器基本结构 （第4章）
  - $s = w_0 + w_1 x_1 + \cdots + w_d x_d = \sum_{i=0}^{d} w_i x_i = \boldsymbol{w}^{\mathrm{T}} \boldsymbol{x} \quad (x_0 = 1)$
  - 非线性处理单元$f(s)$： $\theta(s) = \frac{1}{1+e^{-s}} = \frac{e^s}{e^s+1}$
  - 隐层和节点数目越多，VC维越高，更容易过学习



$$g(\mathbf{x}) = f\left(\sum_j w_{kj} f\left(\sum_i w_{ji} x_i\right)\right)$$



- 前向传播 （第4章）
  - 每层：第一步线性加权组合，第二步非线性变换
  - 基于新特征表示进行判别预测和定量预测
  - “表示学习”（Representation Learning）

- 网络训练（第4章）
  - 误差反向传播算法（BP算法）
  - 原理：误差梯度下降

- 二分类任务
  - 最后一层可用一个节点，是基于新特征的罗杰斯特回归
- 多分类任务
  - 每个节点对应一类，进行“独热”（one-hot）编码
  - 需要进行输出归一化保证和为1
  - 常用归一化指数函数（normalized exponential function），也称软最大函数（softmax 函数）

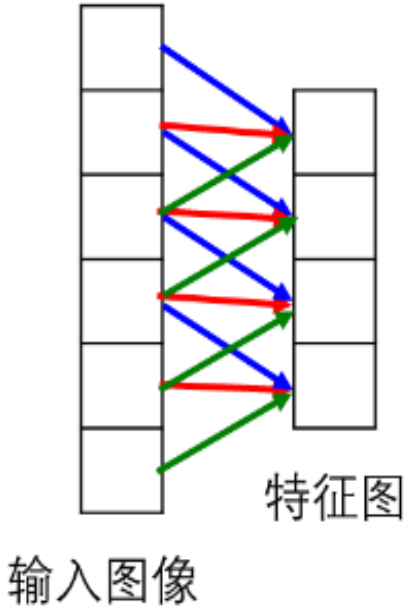
$$o_j = \frac{e^{\boldsymbol{w}_j^T \boldsymbol{x}}}{\sum_{k=1}^{K} e^{\boldsymbol{w}_k^T \boldsymbol{x}}}, \qquad j = 1, \cdots K$$

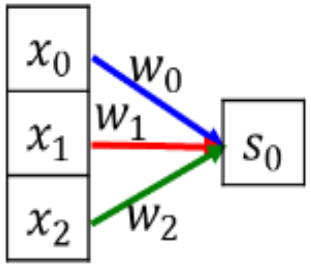
12.3 卷积神经网络（CNN）

- 用于解决图像识别领域的问题
- 原有多层感知器解决图像识别面临欠学习或参数过多问题
- 图像中待识别对象的像素点间存在空间关联关系
- 图像对象中存在多种不变性因素：
  - 位置，大小，变形并不影响字符的属性

12.3.1 卷积层

- 单个卷积核：局部信息提取
- 所有位置扫描，一个卷积核得到一个特征图（Feature Map）
- 多个卷积核称作多个通道（channel），用于不同特征提取
- 一维卷积：相应位置加权求和





$$s_i = \sum_{r=0}^{2} w_r x_{i+r}$$

- 二维卷积（2D Convolution）示例：



原图片

$$*$$

•0	•0	•0
•0	•1	•0
•0	•0	•0

卷积核

$$=$$



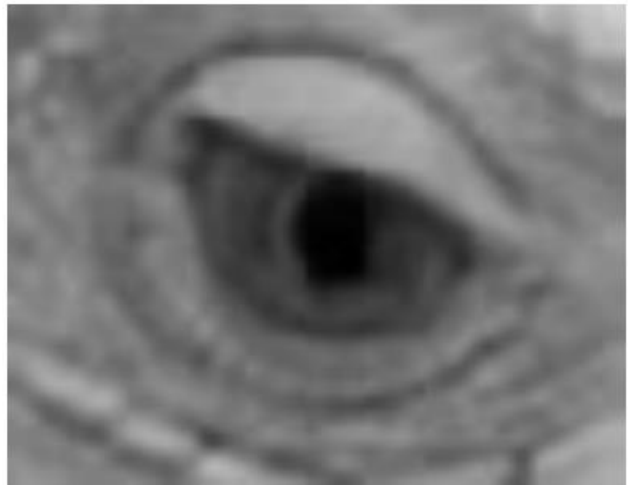
卷积核过滤后图片
（没有区别）

- 二维卷积（2D Convolution）示例：



$$* \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} =$$

取平均=把高频信号去掉



原图片　　平滑卷积核　　卷积核过滤后图片（模糊、低频）

- 二维卷积（2D Convolution）示例：



Original

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} = ?$$

(Note that filter sums to 1)

“details of the image”

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

原图片 + 原图片 - 模糊图片 = ?

- 二维卷积（2D Convolution）示例：

原图片-模糊图片=细节



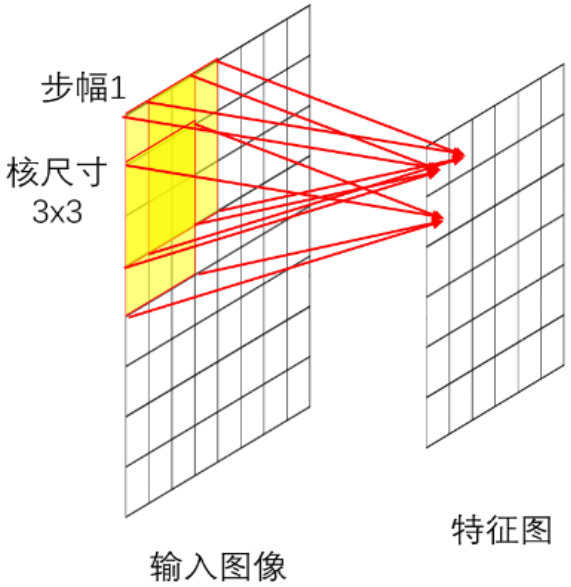
细节是图片的高频特征

- Let’s add it back:

原图片＋细节=细节增强图片



- 超参数（hyper-parameters）
  - 卷积核尺寸
  - 步幅（stride）
  - 边宽或边衬（pad）
- 对训练过程与结果有重要影响



$$s_{ij} = \sum_{c=0}^{2} \sum_{r=0}^{2} w_{cr} x_{i+c,\, j+r}$$

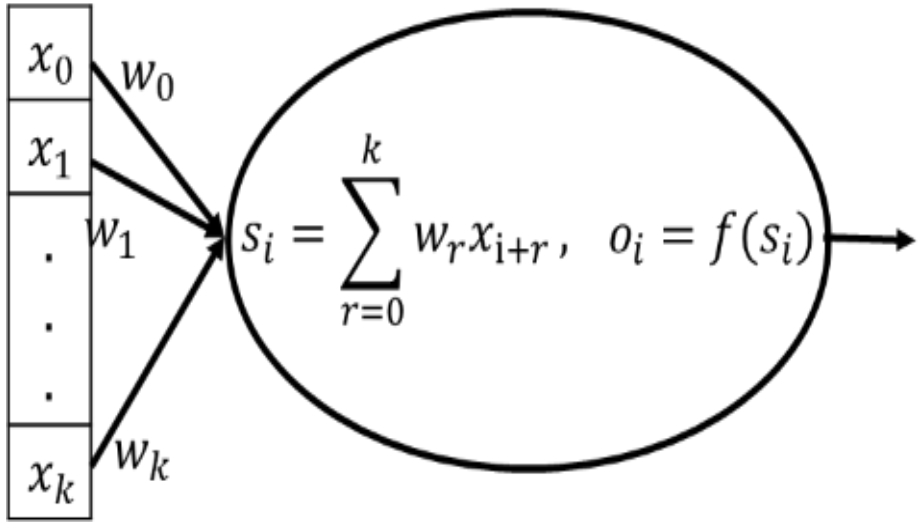
边衬1

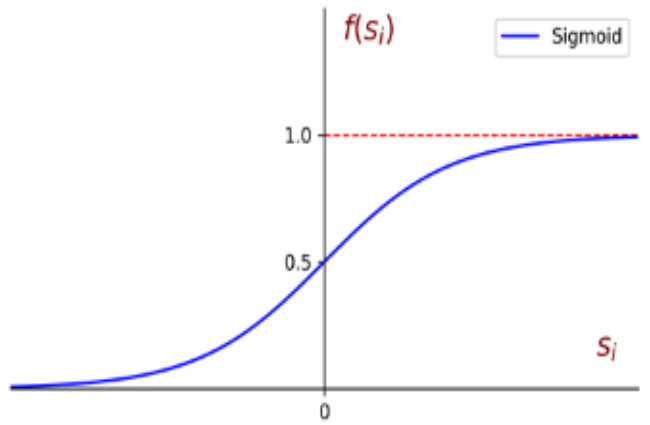
0	0	0	0	0	0	0	0	0	0
0									0
0									0
0									0
0									0
0									0
0									0
0									0
0									0
0	0	0	0	0	0	0	0	0	0

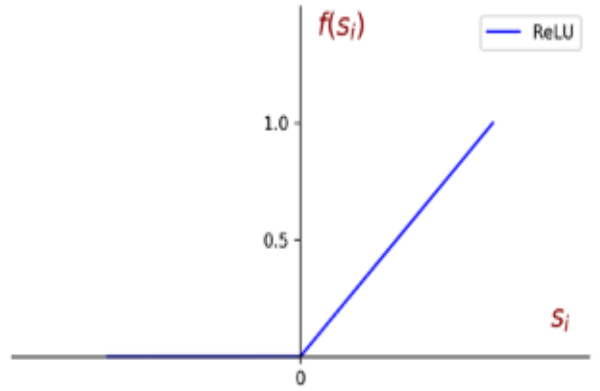
- 权值共享（weight sharing）：一轮扫描中卷积核参数不变
- 前向传播后，通过输出端误差进行反向传播优化
- 多个并行卷积核构成多个通道，提取不同特征
- 图像尺寸$n \times n$，卷积核尺寸$k \times k$，步幅为s，边衬为p且对称，则特征图维数：

$$\left\lfloor \frac{n + 2p - k}{s} + 1 \right\rfloor$$

- 挤压（squashing）：对卷积结果进行非线性运算
- 非线性激活函数
  - 传统S型函数（sigmoid） 或双曲正切函数（tanh）
  - 常用ReLU函数：矫正线性单元（Rectified Linear Unit）







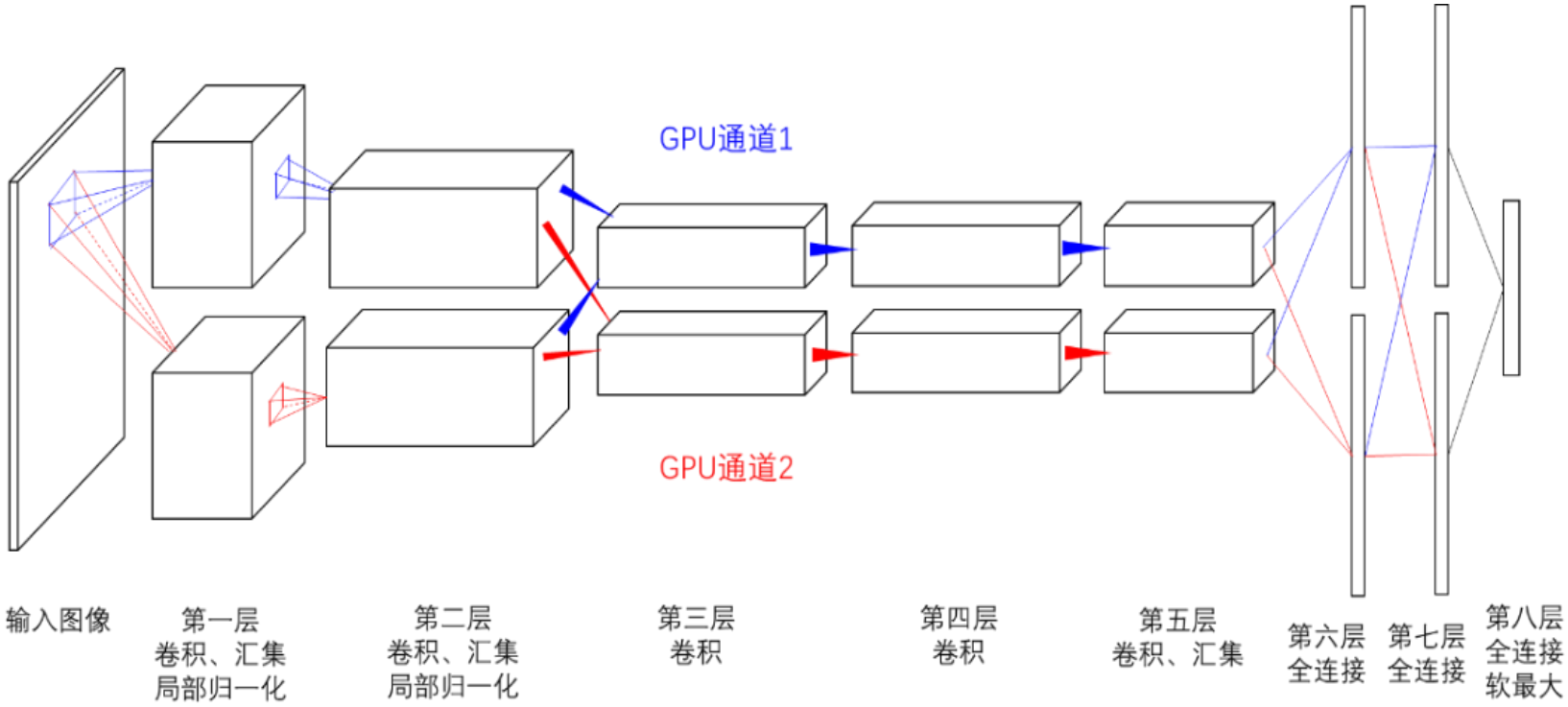
12.3.2 汇集（池化）

- 汇集（“池化”）：对卷积后特征图进行降采样
  - 最大汇集（max pooling）：选局部区域最大值
  - 平均汇集（Average pooling）：选区域数值平均值
- 两个参数：汇集区域大小和步幅



12.3.3 深层卷积神经网络

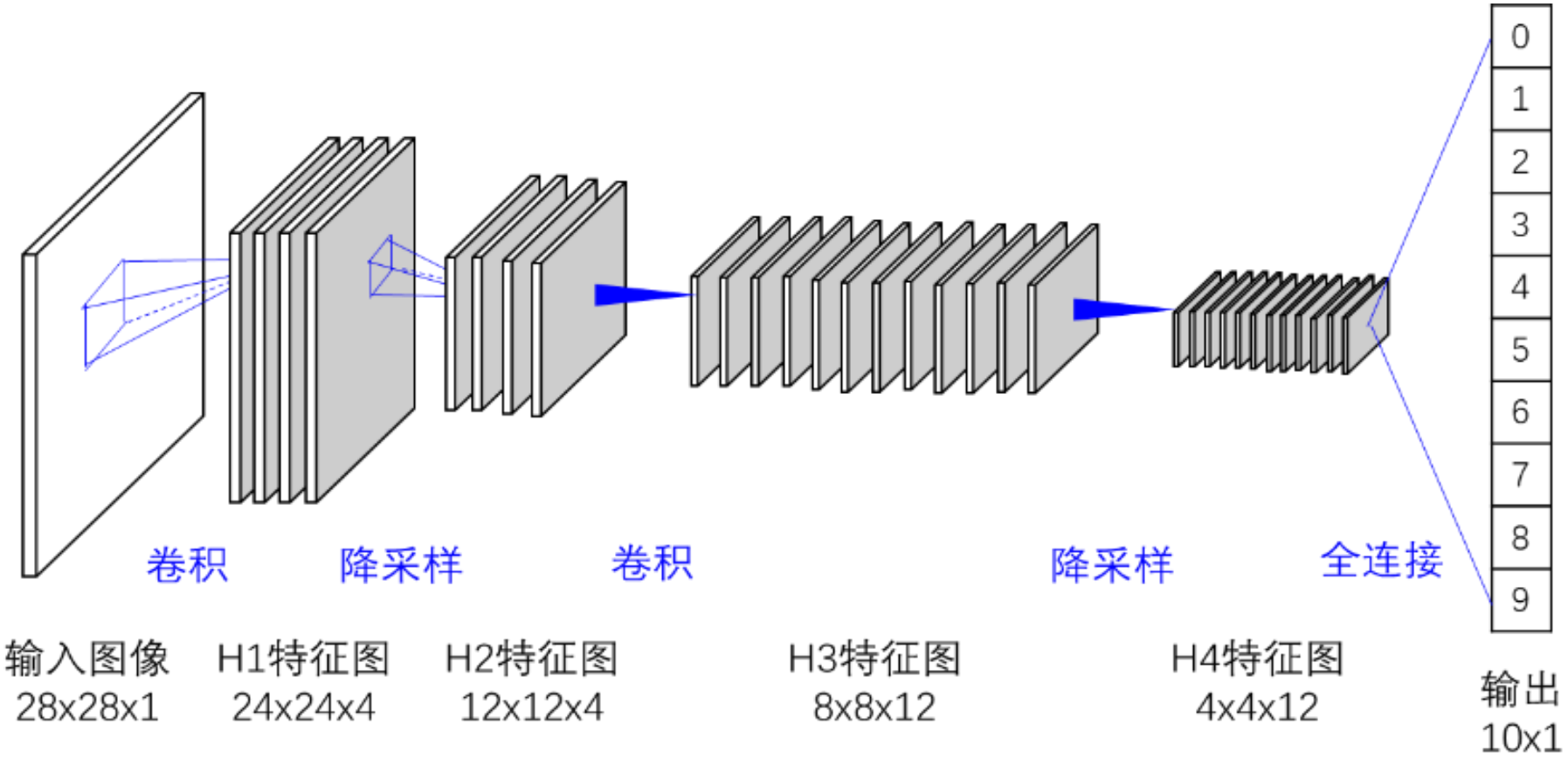
- AlexNet
  - 使用局部感受野、权值共享、最大汇集等措施减少参数量
  - 自由参数：$\sim 6.1 \times 10^7$
  - 数据扩增（data augmentation）：利用预处理增加训练样本数目



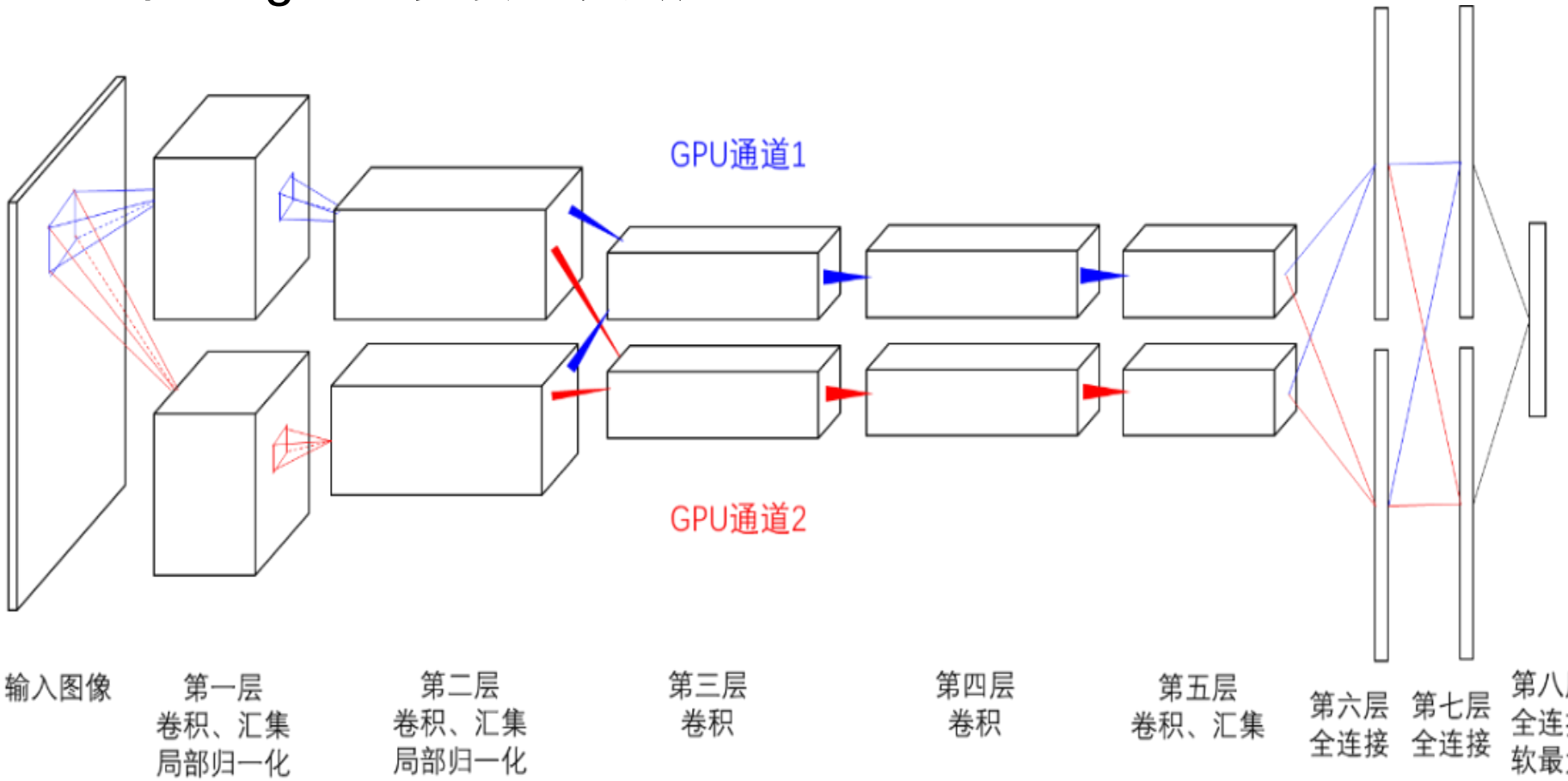
12.3.4 卷积神经网络的演化和几个代表性模型

- 1990年：LeNet-1
- 2012年：AlexNet
- 2013年：ZFNet
- 2014年：VGG 网络
- 2014年：GoogLeNet
- 2015年：残差网络（ResNet）
- 2017年：密集网络（DenseNet）
- 2018年：多尺度密度网络（MSDNet）
- 2019年：EfficientNet
- 2020年：Vision Transformers（ViT）
- 2021年：自适应卷积网络

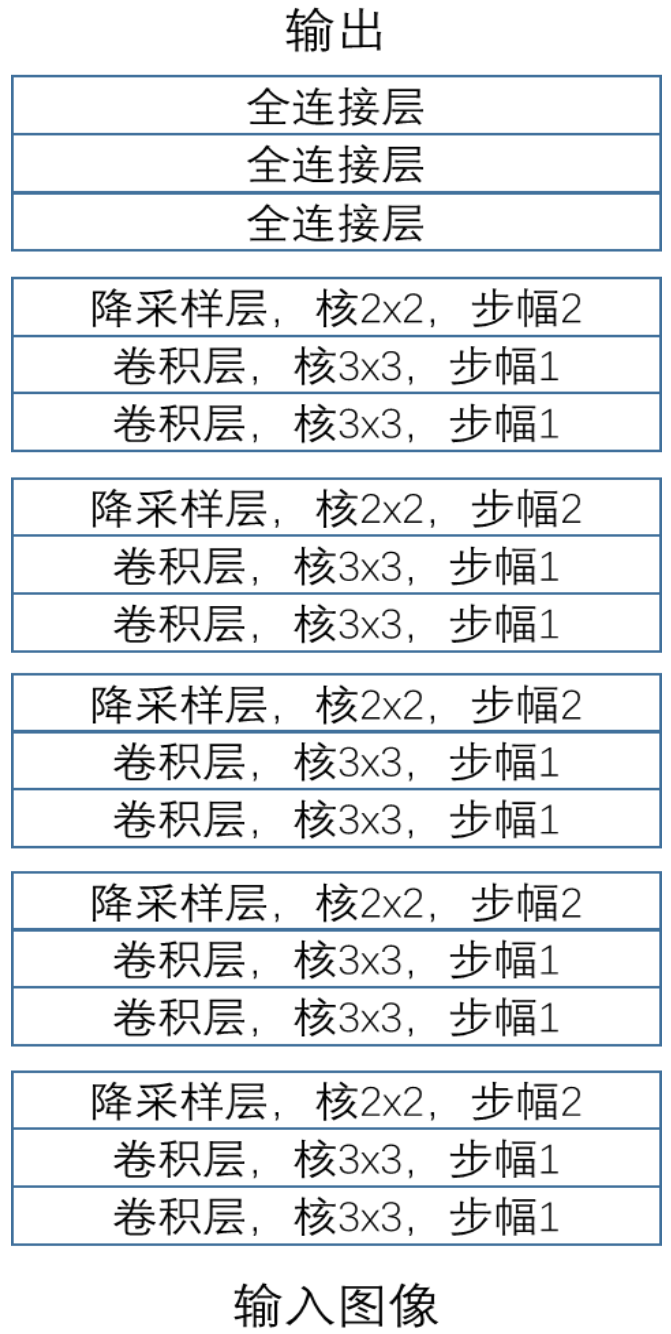
- LeNet-1
  - 针对手写字符问题设计，后被支持向量机超出（6.5.3小节）



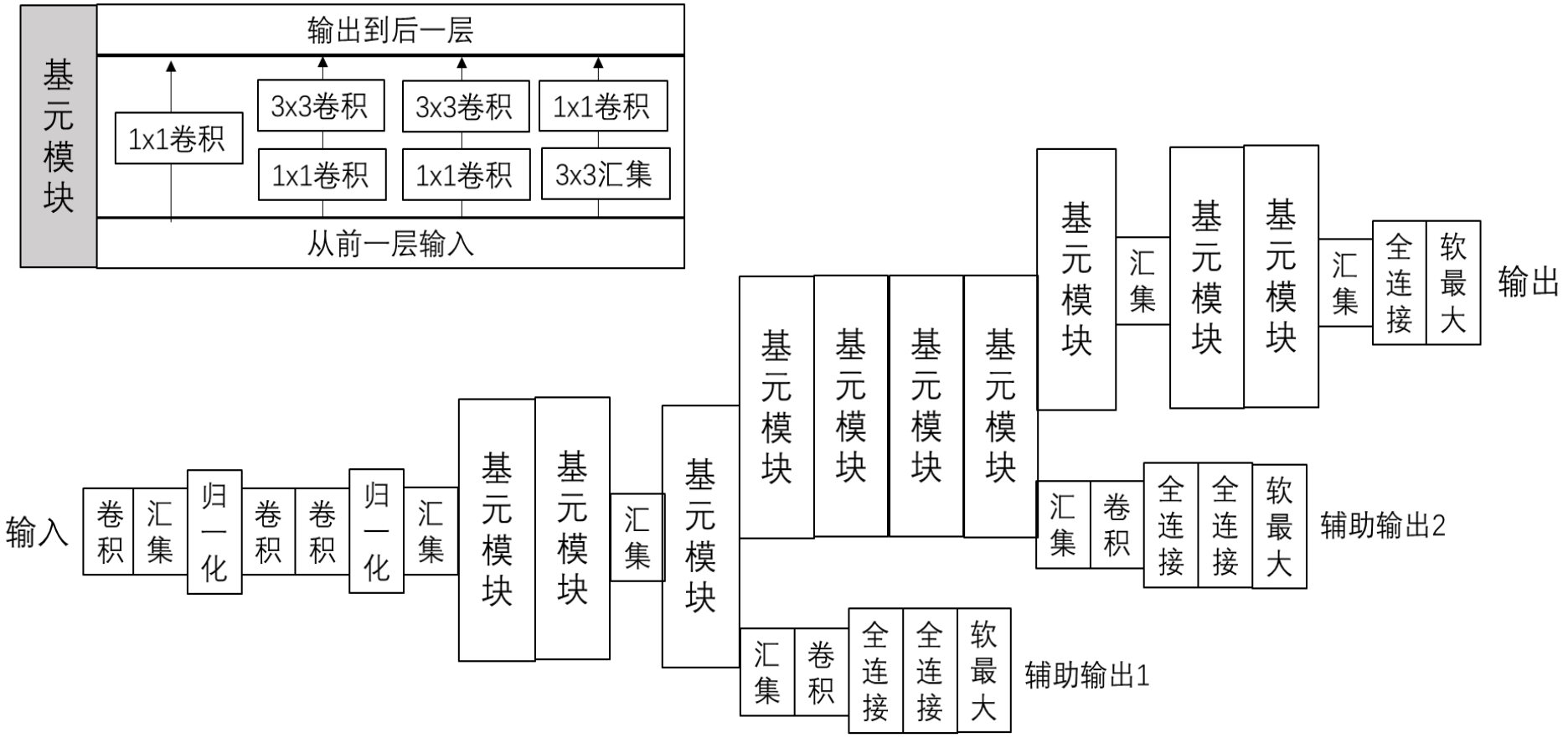
- AlexNet
  - 2012年ImageNet竞赛大突破



- VGG网络
  - 使用了较小卷积核尺寸和降采样区域
  - 网络提升到最多16~19层

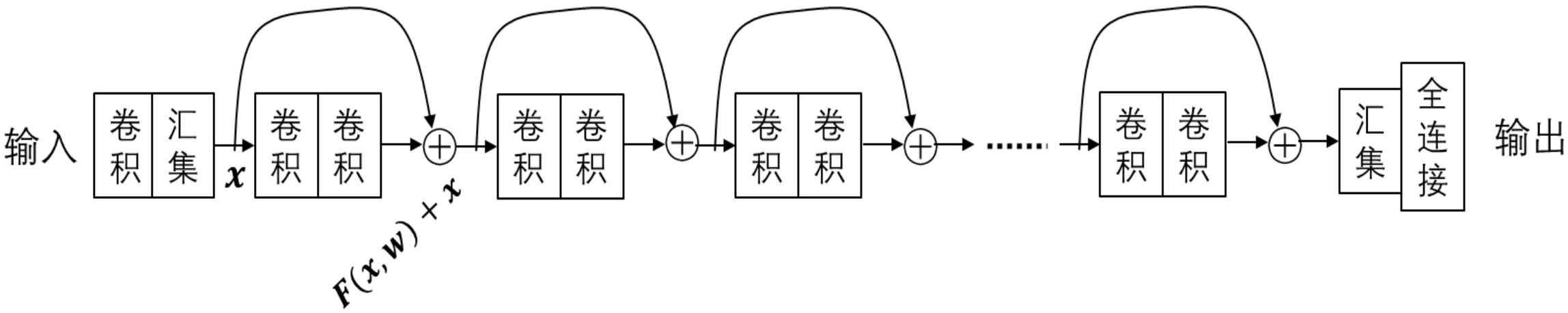


- GoogLeNet
  - 用多个“起始单元”（inception）的小网络模块构成大网络
  - 在基元中采用了多个1x1的卷积核，实现降维和减少计算量
  - 在中间设置两处输出，作为训练辅助

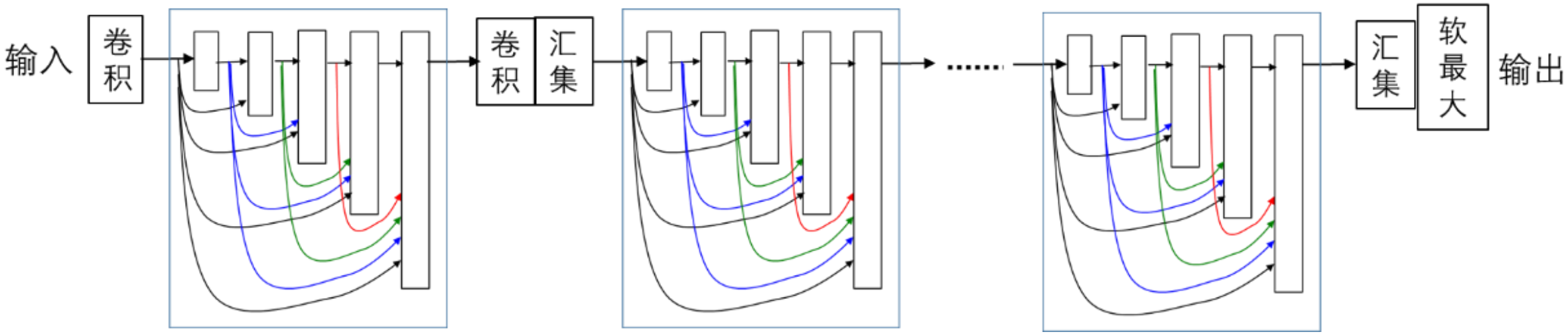


- 残差网络（ResNet）
  - 相邻层合并为一个组合单元，每个单元设一个跨过几层网络的短路连接
  - $\boldsymbol{x}$ 是本单元的输入，单元格内局部网络映射$\boldsymbol{F}(\boldsymbol{x},\boldsymbol{w})$，是除信号自身外的一个“残差”（residual），则实际输出$\boldsymbol{y}$：

$$\boldsymbol{y} = \boldsymbol{F}(\boldsymbol{x},\boldsymbol{w}) + \boldsymbol{x}$$



- 密集网络（DenseNet ）
  - 上一层得到的特征图不但输出到下一层，而是形成了跨层的密集连接
  - 密集区块（Dense Block）：用较少的卷积核，并且大量是1x1 卷积核
  - 进一步拓展为MSDNet：二维的神经网络阵列，并设有中间输出



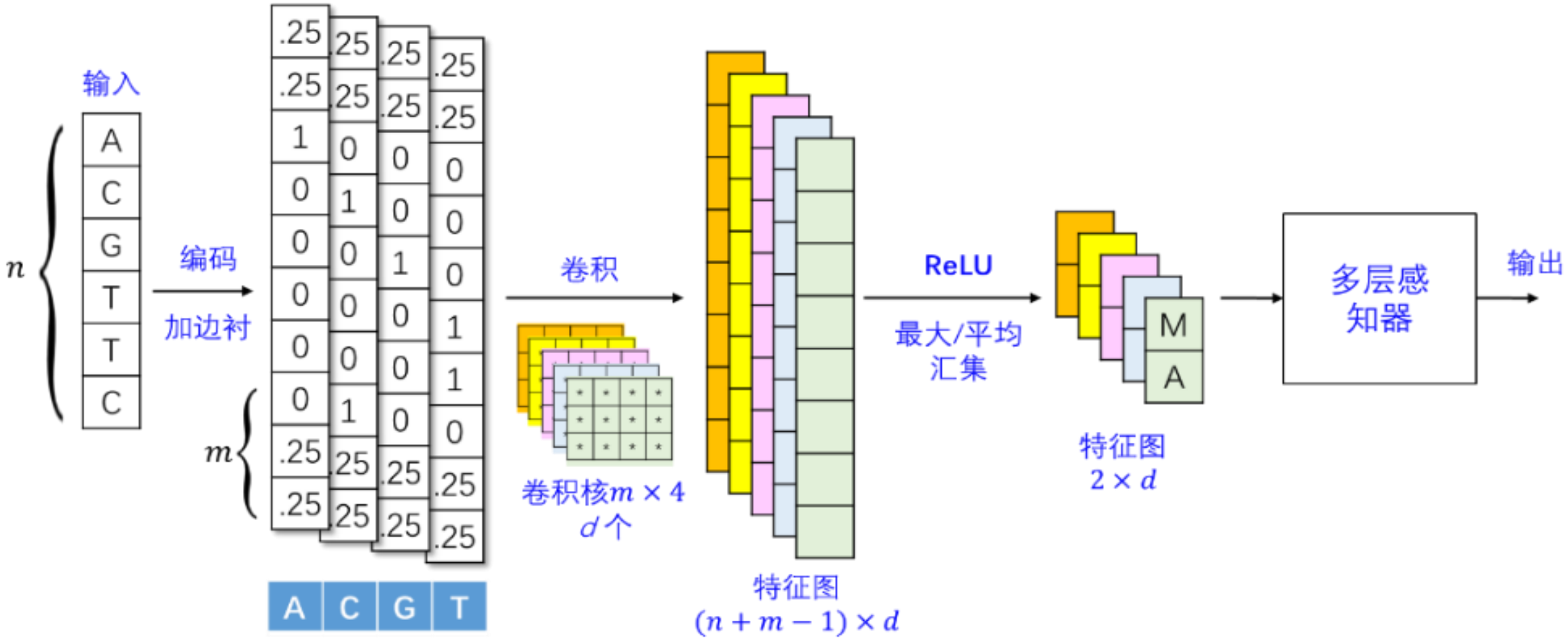
12.3.5 卷积神经网络在非图像数据上的应用举例

- DeepBind：预测DNA序列与蛋白质结合能力
- 编码A，C，G，T和不确定碱基N为4维向量
- 使用一维，四个通道的卷积核学习模体
- 输出函数：

$$f(s) = \mathrm{net}_W\left(\mathrm{pool}\left(\mathrm{rect}_b\left(\mathrm{conv}_M(s)\right)\right)\right)$$

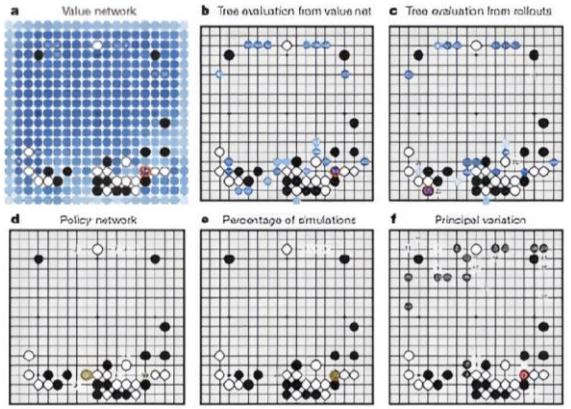
- s是编码后的输入序列， $\mathrm{conv}()$表示卷积层， $\mathrm{pool}(\cdot)$ 表示汇集运算， $\mathrm{rect}(\cdot)$表示ReLU函数，$\mathrm{net}(\cdot)$表示全连接多层感知器

- 核函数反应结合序列每个位置的相对重要性，称作“突变图”



- CNN的应用
  - AlphaGo







nature

Explore content · About the journal · Publish with us

nature › articles › article

Published: 27 January 2016

Mastering the game of Go with deep neural networks and tree search

David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel & Demis Hassabis

Nature **529**, 484–489 (2016) | Cite this article

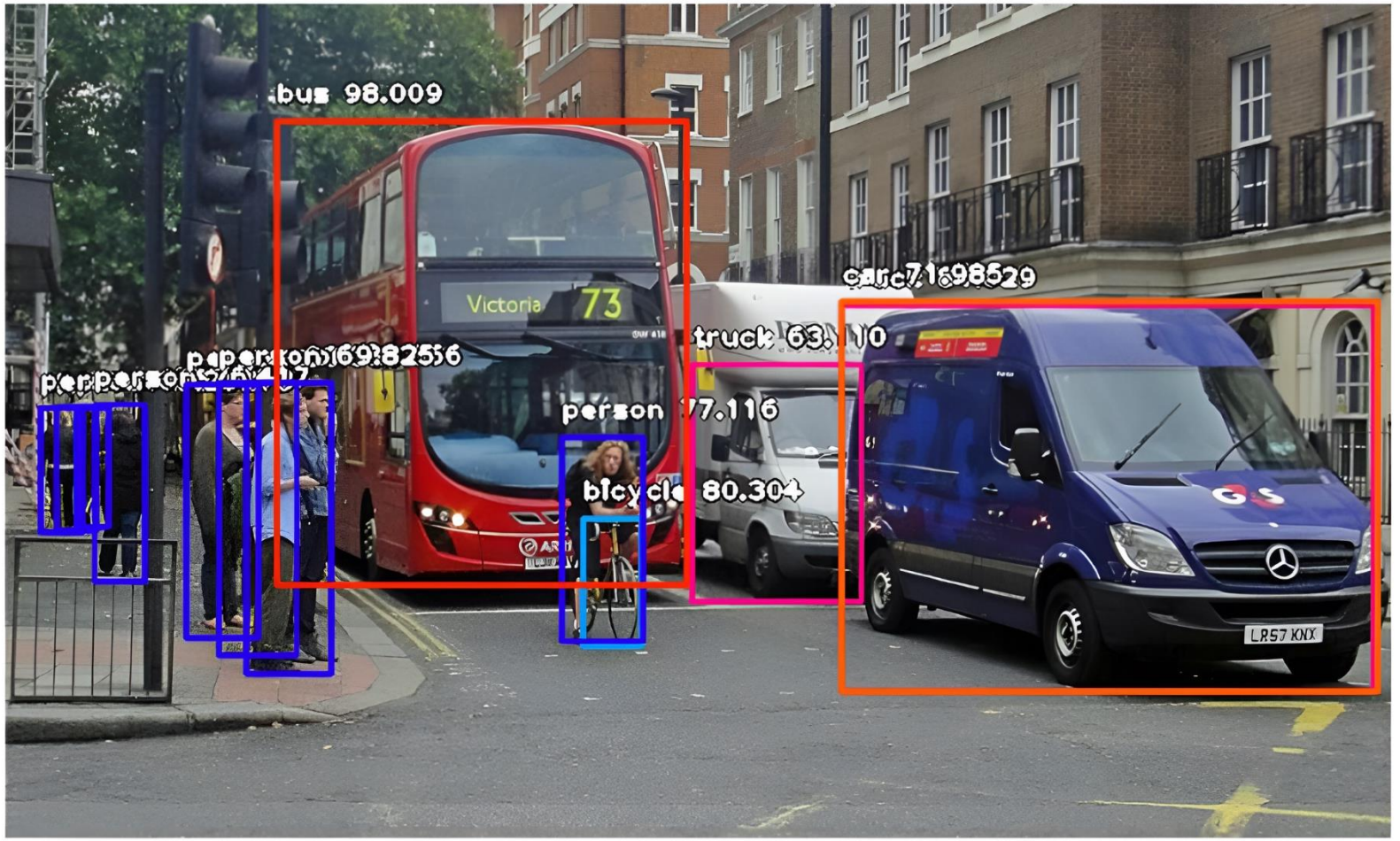
459k Accesses | **7902** Citations | **3093** Altmetric | Metrics

Abstract

The game of Go has long been viewed as the most challenging of classic games for artificial intelligence owing to its enormous search space and the difficulty of evaluating board positions and moves. Here we introduce a new approach to computer Go that uses 'value networks' to evaluate board positions and 'policy networks' to select moves. These deep neural networks are trained by a novel combination of supervised learning from human expert games, and reinforcement learning from games of self-play. Without any lookahead search, the neural networks play Go at the level of state-of-the-art Monte Carlo tree search programs that simulate thousands of random games of self-play. We also introduce a new search algorithm that combines Monte Carlo simulation with value and policy networks. Using this search algorithm, our program AlphaGo achieved a 99.8% winning rate against other Go programs, and defeated the human European Go champion by 5 games to 0. This is the first time that a computer program has defeated a human professional player in the full-sized game of Go, a feat previously thought to be at least a decade away.

The input to the policy network is a 19 x 19 x 48 image stack consisting of 48 feature planes. Thefirst hidden layer zero pads the input into a 23 x 23 image, then convolves k filters of kernel size 5x 5 with stride 1 with the input image and applies a rectifier nonlinearity. Each of the subsequenthidden layers 2 to 12 zero pads the respective previous hidden layer into a 21 x 21 image, thenconvolves k filters of kernel size 3 x 3 with stride 1, again followed by a rectifier nonlinearity. Thefinal laver convolves 1 filter of kernel size 1 x 1 with stride 1. with a different bias for each positionand applies a softmax function. The match version of AlphaGo used k= 192 filters; Fig. 2b andExtended Data Table 3 additionally show the results of training with k= 128, 256 and 384 filters.

- CNN的应用
  - 目标检测


bus 98.009
Victoria
73
truck 63.
person
77.116
bicycl
80.304
LR57 KNX

- CNN的应用
  - 沥青路面裂缝检测和分类

Automation in Construction 160 (2024) 105297

Contents lists available at ScienceDirect

Automation in Construction

journal homepage: www.elsevier.com/locate/autcon

Evaluation and optimisation of pre-trained CNN models for asphalt pavement crack detection and classification

Sandra Matarneh [a], Faris Elghaish [b,*], Farzad Pour Rahimian [c,*], Essam Abdellatef [d], Sepehr Abrishami [e]

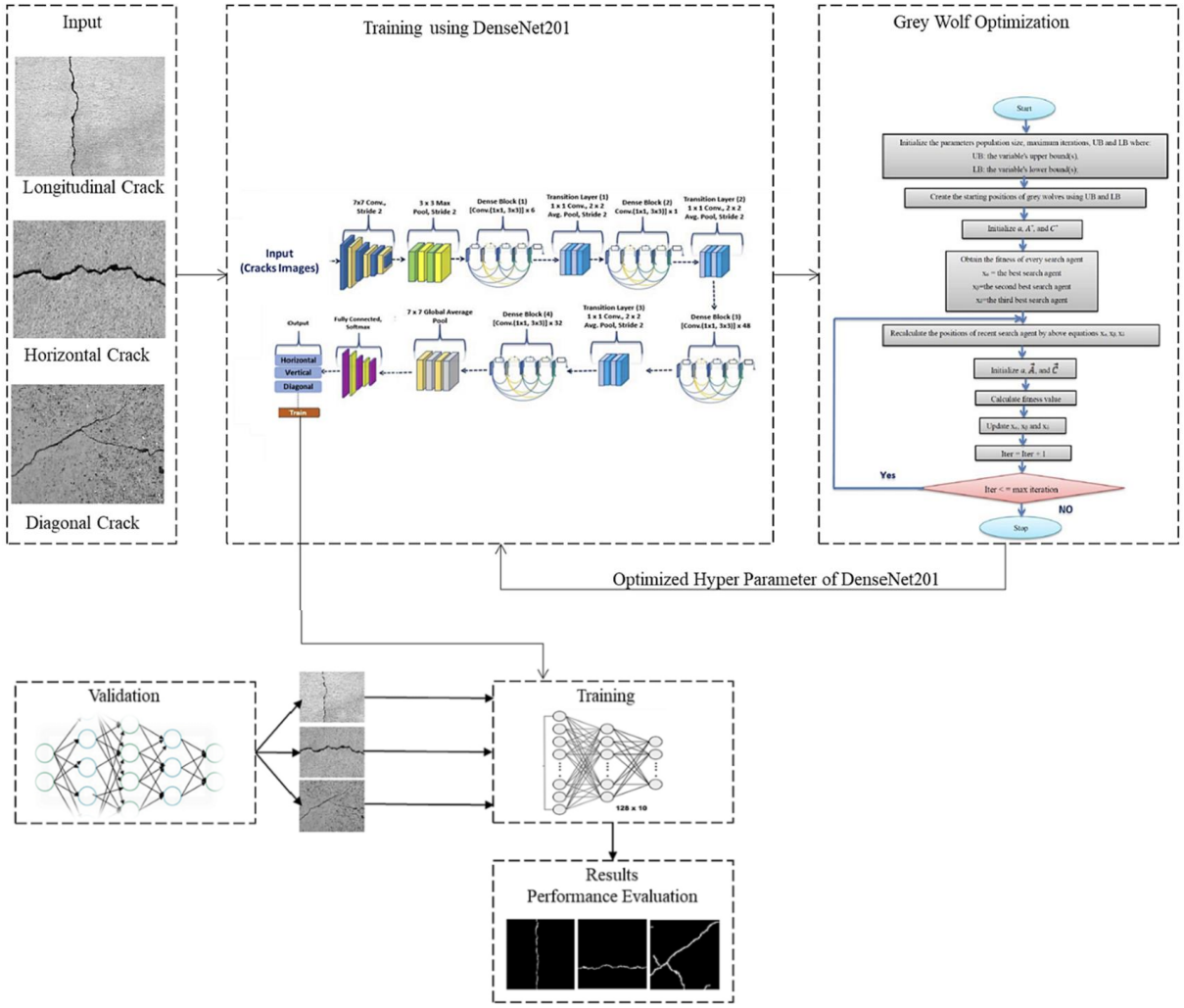
[a] Faculty of Engineering, Al Ahliyya Amman University, Jordan
[b] School of Natural and Built Environment, Queen's University Belfast, Belfast, UK
[c] School of Computing, Engineering & Digital Technologies, Teesside University, Middlesbrough, UK
[d] Department of Electrical Engineering, Faculty of Engineering, Sinai University, El-Arish 45511, Egypt
[e] School of Civil Engineering and Surveying, University of Portsmouth, Portsmouth, UK

ARTICLE INFO

Keywords:
Asphalt pavement
Cracks classification
CNN
OpDenseNet201
Optimisation
Deep learning
Pre-trained models

ABSTRACT

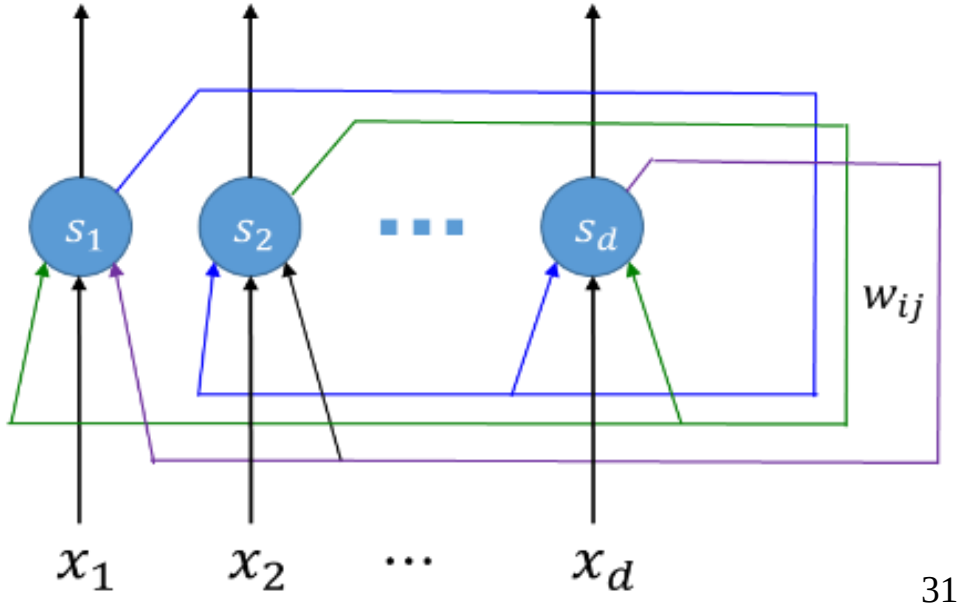
This study explored the performance of ten pre-trained CNN architectures in detecting and classifying asphalt pavement cracks from images. A comparison of eight optimisation techniques led to developing an optimised pre-trained CNN model tailored for crack classification, with DenseNet201 emerging as the most effective, closely followed by ShuffleNet and ResNet101. Conversely, VGG16 exhibited notably lower accuracy among the models evaluated. Through the application of diverse feature selection techniques as optimisers, DenseNet201 consistently outperformed others, followed by DarkNet19 and Xception. Despite employing different optimisers, VGG16 and VGG19 consistently demonstrated inferior performance. The research introduced a novel approach utilising the DenseNet201 model and the GWO optimiser for asphalt pavement crack classification, validated against various CNN models. Its robustness was verified by testing against images contaminated with differing levels and types of noise, yielding promising outcomes. Results underscore the method's potential for accurately detecting diverse crack types, implying applicability in real-world scenarios.



12.4 循环神经网络（RNN）

- 霍普菲尔德（Hopfield）网络
  - 基本原理：联想记忆（associative memory）或内容索引的记忆（content-addressable memory）
  - 每个神经元s_i由其他神经元s_j及连接关系w_{ij}共同决定

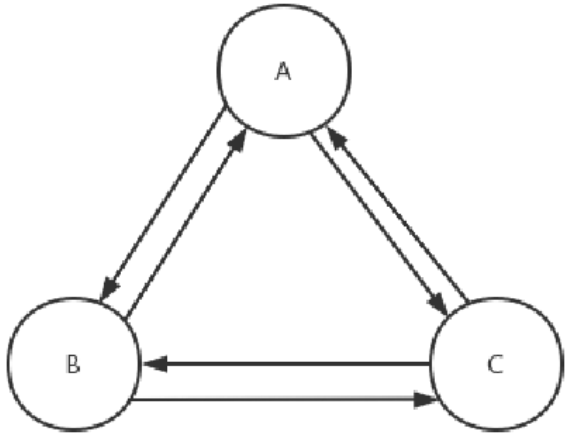
- $$s_i = \begin{cases} +1 & \text{若} \sum_j w_{ij} s_j \geq \theta_i \\ -1 & \text{其他} \end{cases}$$



12.4.1 Hopfield神经网络

Hopfield神经网络是一种比较特殊的网络，它不像一般的神经网络那样有输入层和输出层，并且通过训练来改变神经网络中的参数，最终实现预测、识别等功能。Hopfield网络只有一群**神经元节点**，所有节点之间相互连接。

只有三个神经元的Hopfield网络为例，令这三个神经元为A，B，C，其图形如下所示：



其中箭头表示从一个神经元到另一个神经元相连，每个箭头都表示相应的权重，为一个数，可以用ω_{ij}来表示（神经元自己和自己也有连接，数值始终是0）

以矩阵形式表示各个神经元之间的连接：

	A	B	C
A	0	ω_{AB}	ω_{AC}
B	ω_{BA}	0	ω_{BC}
C	ω_{CA}	ω_{CB}	0

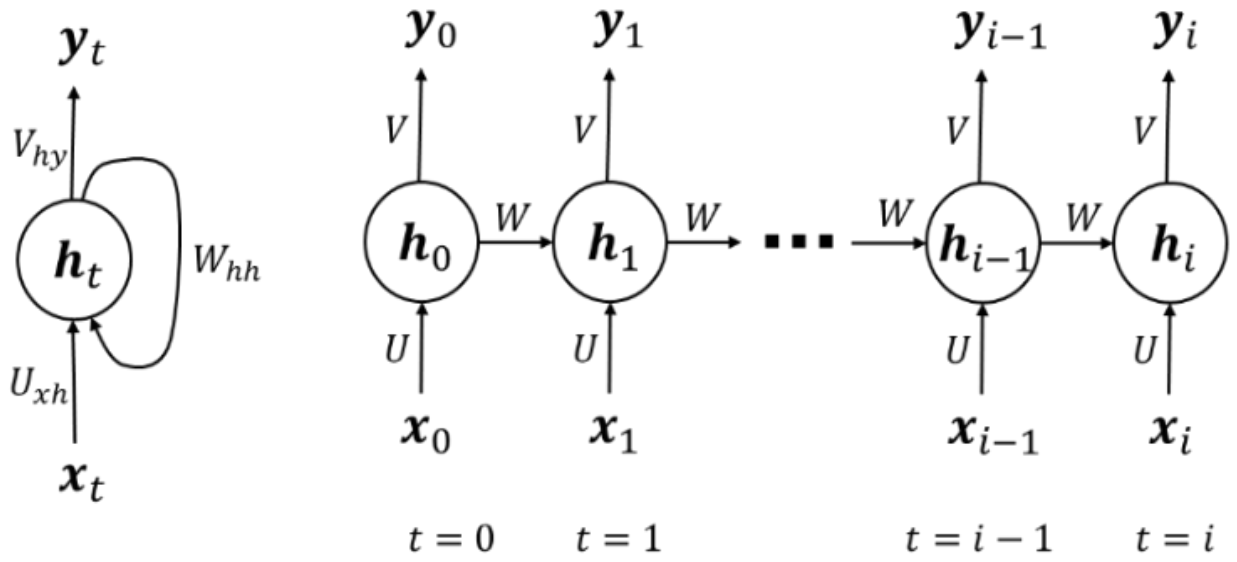
12.4.2 循环神经网络

- Hopfield网络：

$$\boldsymbol{h}_t = f_{\boldsymbol{w}}(\boldsymbol{h}_{t-1}, \boldsymbol{x}_0)$$

- 对于时间序列样本，我们希望建立随时间逐步输入的模型
- 循环神经网络：

$$\boldsymbol{h}_t = f_{\boldsymbol{w}}(\boldsymbol{h}_{t-1}, \boldsymbol{x}_t)$$



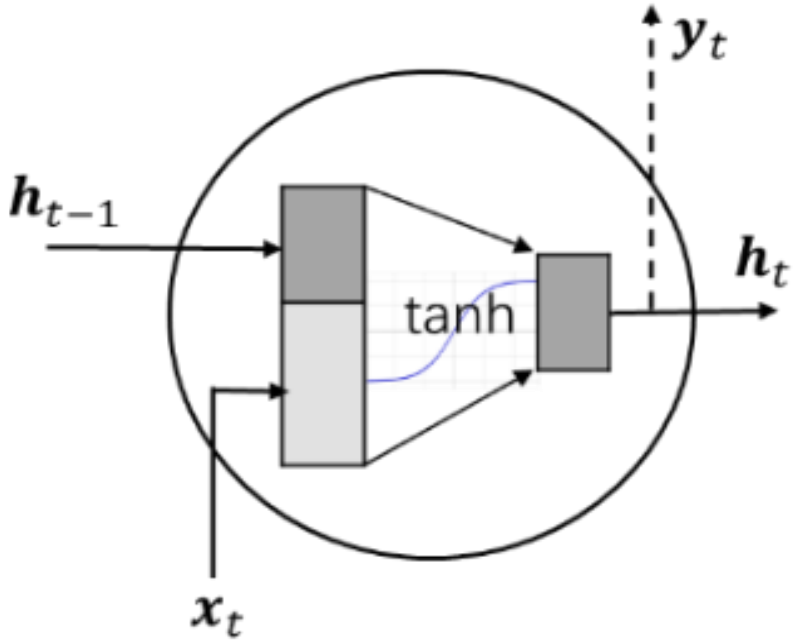
- 基本RNN运算函数：

$$\boldsymbol{h}_t = \tanh(W_{hh}\boldsymbol{h}_{t-1} + U_{xh}\boldsymbol{x}_t)$$

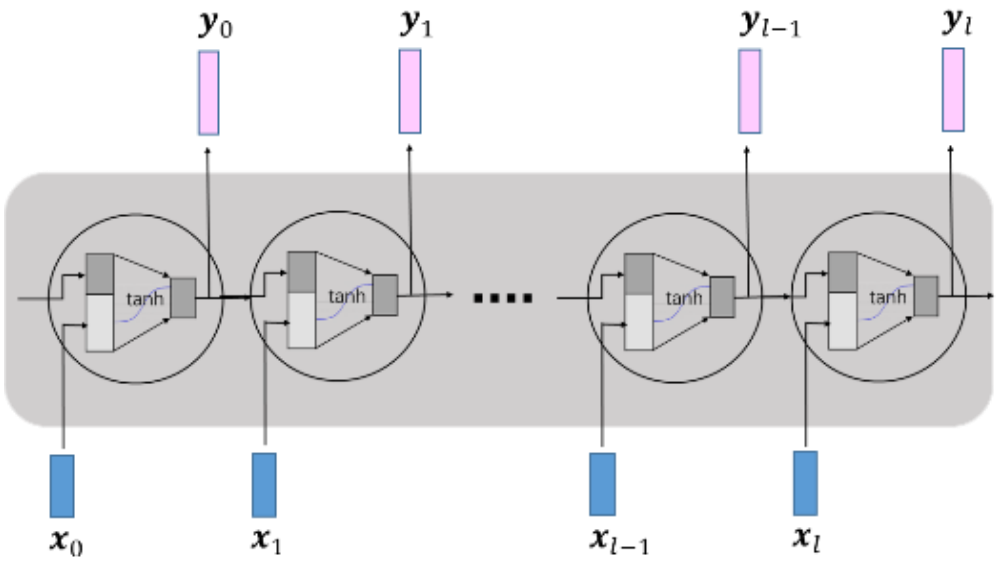
  - W_{hh}：自身的权值矩阵
  - U_{xh}：输入向量到神经元节点的权值矩阵
  - $\boldsymbol{h}_t$当前时刻状态
  - $\boldsymbol{h}_{t-1}$前一时刻状态
- t 时刻网络输出

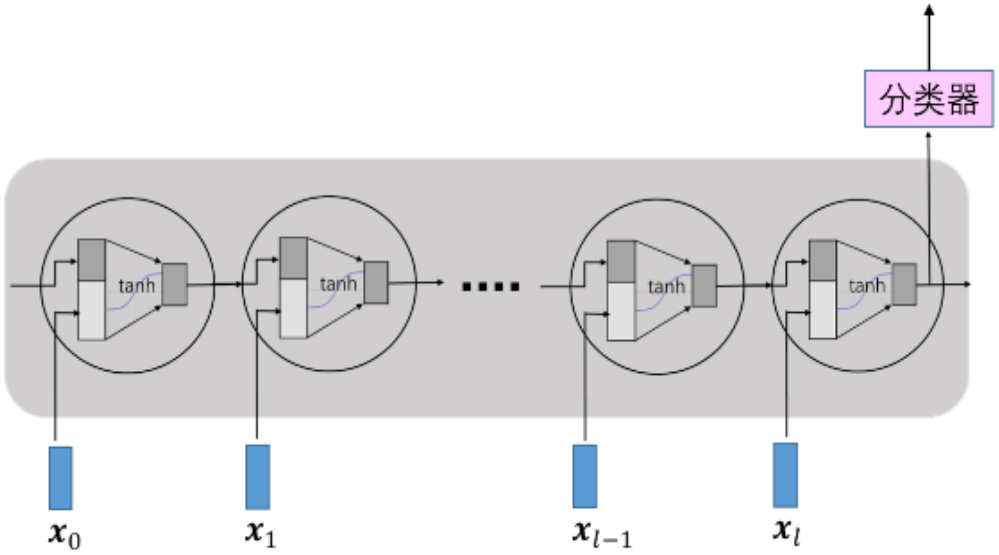
$$\boldsymbol{y}_t = V_{hy}\boldsymbol{h}_t$$

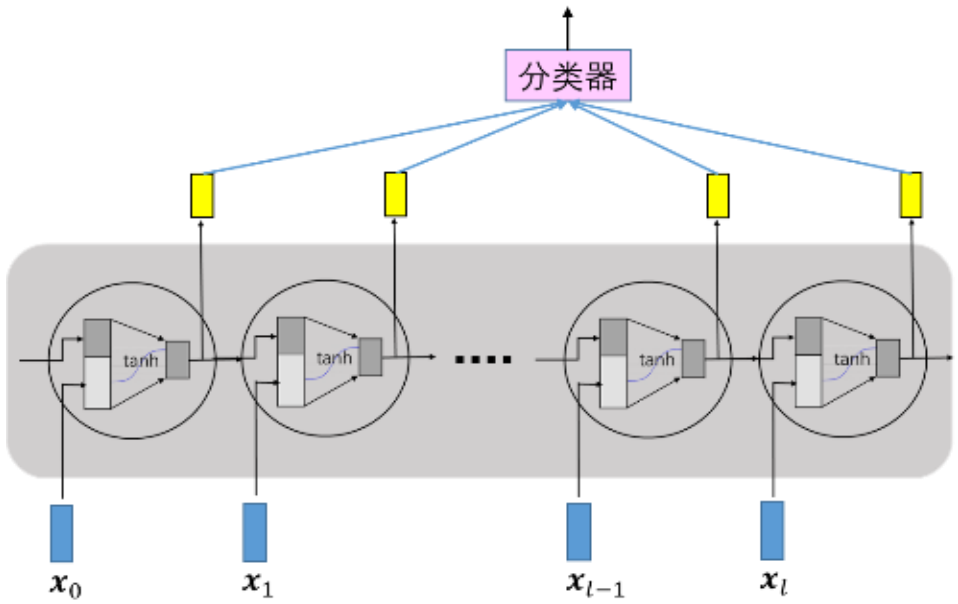
- RNN是有时间过程的前馈神经网络

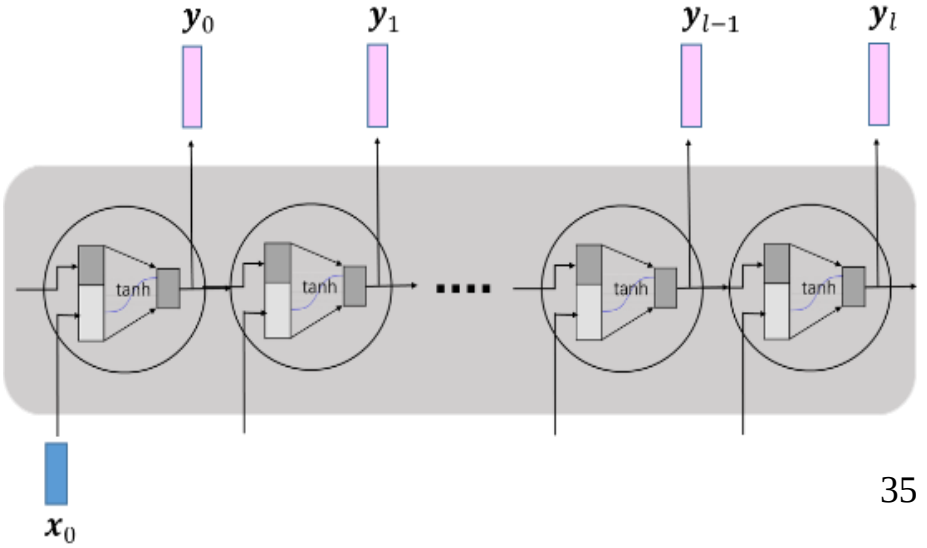


- RNN几种应用形式


y_0
y_1
y_{l-1}
y_l
tanh
x_0
x_1
x_{l-1}
x_l


分类器
tanh
x_0
x_1
x_{l-1}
x_l


分类器
tanh
x_0
x_1
x_{l-1}
x_l


y_0
y_1
y_{l-1}
y_l
tanh
x_0

- RNN的训练
  - 正向传播时权值矩阵相同
  - 每个时刻计算误差
  - 权值更新时对误差求和
  - 时间反向传播（Back-propagation through time）
- 用RNN建立语言模型
  - 对单词进行编码
  - 用起始单词创建句子
- CNN-RNN复合神经网络模型：从图像产生语言描述

12.5 长短时记忆模型（LSTM）

- 循环神经网络按照时间顺序依次传递，是一种短时记忆
- 自然语言及其他时间序列信号中，某些信息不限于相邻时刻，有较长远的影响
- 长短时记忆神经网络模型LSTM（Long short-term memory）
  - 1997年已提出
- LSTM有两套随时间传递的状态向量
  - $\boldsymbol{h}_t$为隐状态，与RNN中的神经网络状态类似
  - C_t为记忆状态，跨时间单元传递

- 四个控制通道，称作“门”（gate）
  - 输出门：更新状态变量

$$\boldsymbol{o}_t = \sigma(W_o \boldsymbol{s}_t + b_o)$$
$$\boldsymbol{h}_t = \boldsymbol{o}_t \otimes \tanh(C_t)$$

  - 遗忘门：

    生成遗忘向量$\boldsymbol{f}_t$，部分遗忘前记忆向量C_{t-1}

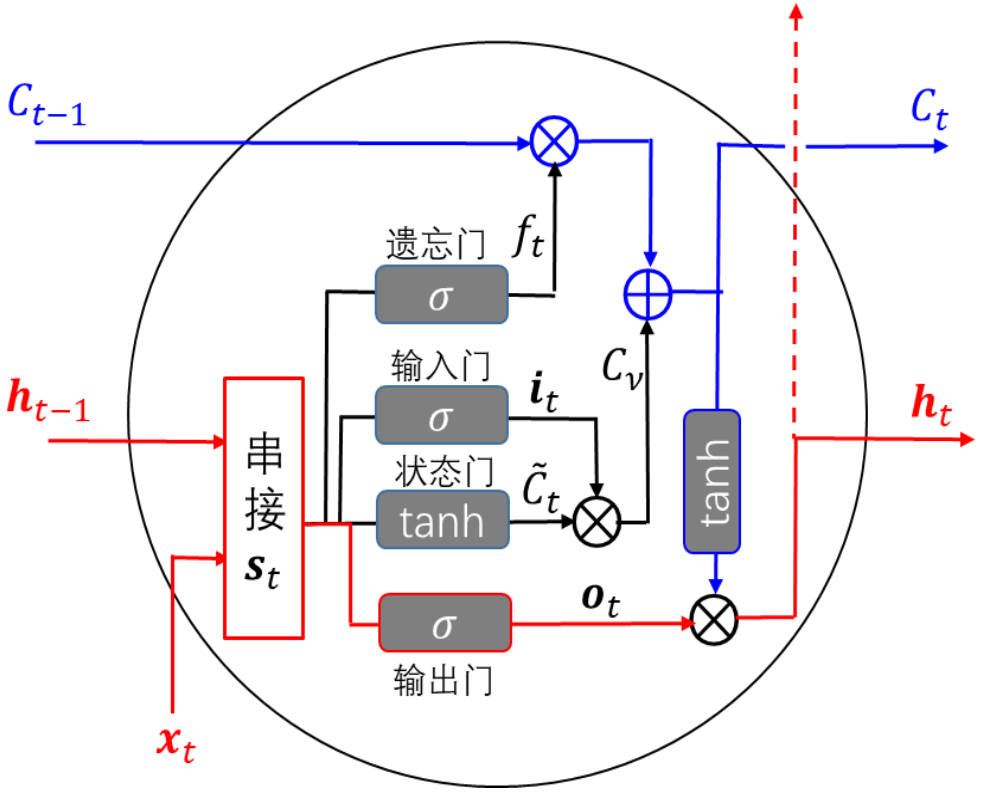
$$\boldsymbol{f}_t = \sigma(W_f \boldsymbol{s}_t + b_f)$$

  - 内部状态门：生成当前状态

$$\tilde{C}_t = \tanh(W_c \boldsymbol{s}_t + b_c)$$

  - 输入门：对当前记忆进行控制

$$\boldsymbol{i}_t = \sigma(W_i \boldsymbol{s}_t + b_i)$$



- 新记忆向量：由内部状态门和输出门共同决定

$$C_v = \boldsymbol{i}_t \otimes \tilde{C}_t$$

- LSTM的训练与RNN基本相同：时间反向传播算法
- RNN回传需要与权值矩阵相乘，可能会梯度爆炸或消失
- LSTM的记忆状态有利于梯度回传，是一条“高速公路”
- 跨时间单元传递的记忆通道与卷积神经网络中ResNet或DenseNet的思想具有一定的共性

12.6 自编码器、限制性玻尔兹曼机与深度信念网络

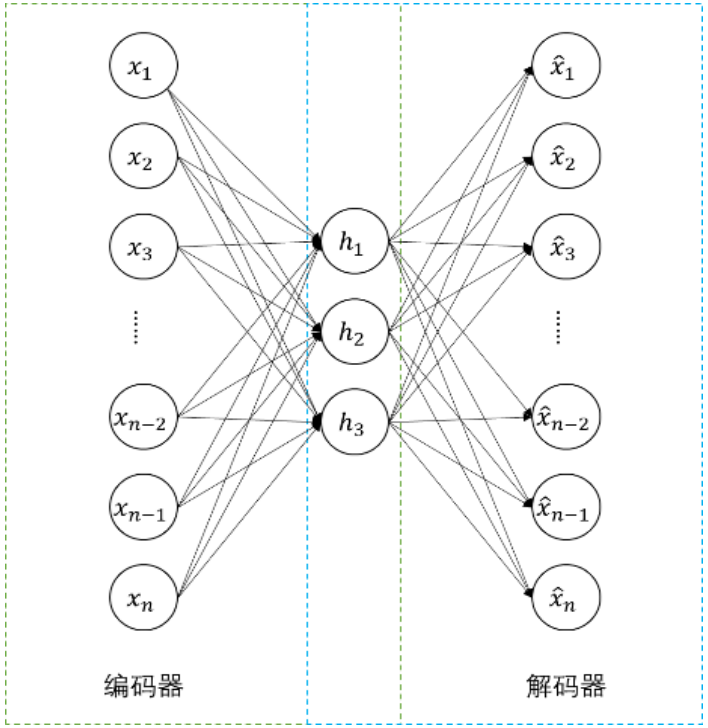
12.6.1 自编码器

- 自我映射：多层感知器的输入输出设置为一样
- 自编码器（Auto-Encoder，缩写为AE）：

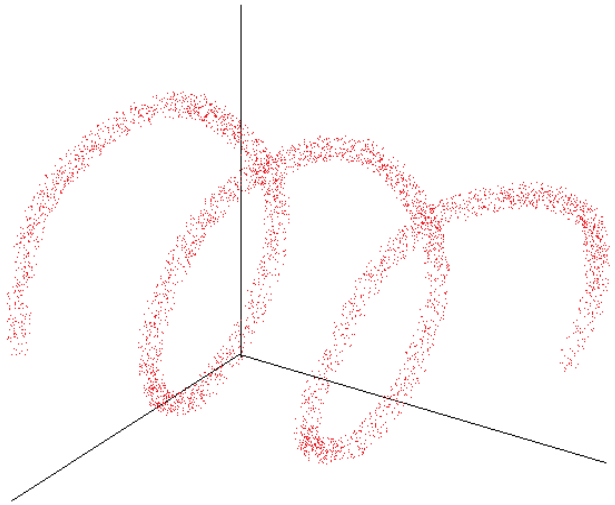
$$g\big(f(\boldsymbol{x})\big) = \boldsymbol{x}$$

  - $f(\boldsymbol{x})$：隐层输出，对样本的低维表示
  - $g(\cdot)$：解码
- 隐节点进行信息编码和压缩
- 目标函数：最小化误差平方和

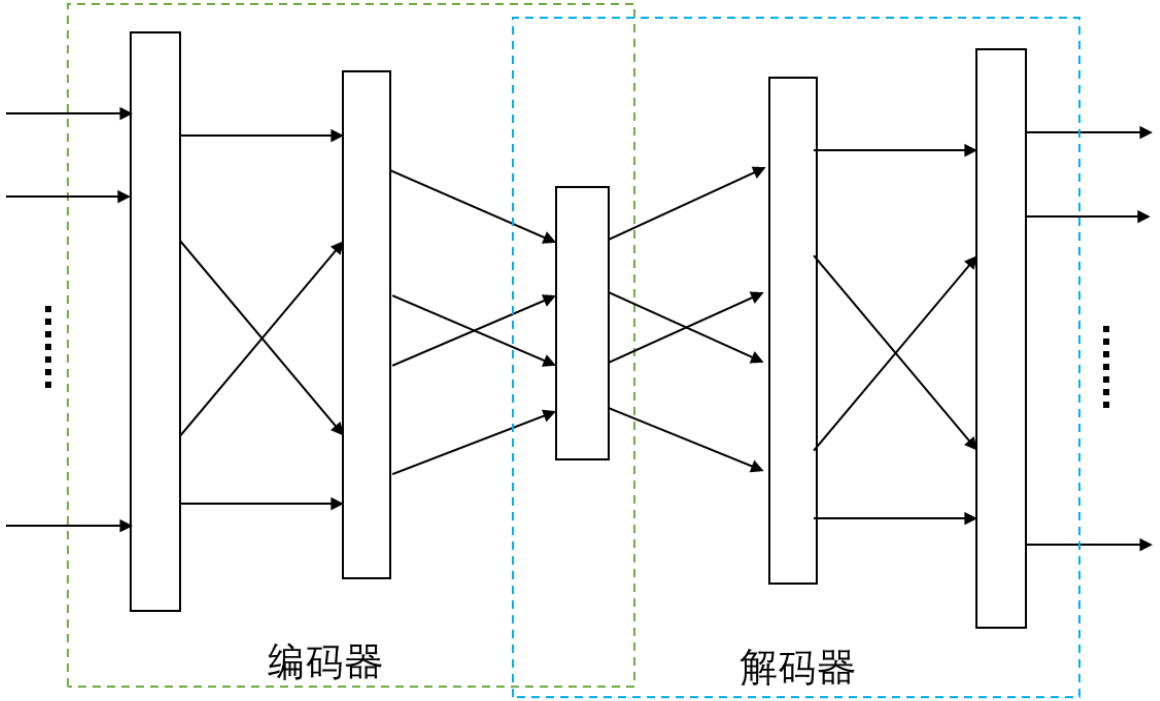
$$\min L\left(\boldsymbol{x}, g\big(f(\boldsymbol{x})\big)\right) = \frac{1}{N}\sum_{i=1}^{N}\left\|g\big(f(\boldsymbol{x}_i)\big) - \boldsymbol{x}_i\right\|^2$$



- 若采用线性函数，则自编码器等价于主成分分析，隐层编码为前几个主成分
- 采用如Sigmoid的非线性传递函数时，可看作是用神经网络实现非线性主成分分析
- 训练仍可采用误差反向传播算法
- 学习中不包含对样本分类或其他特性的预测，是非监督学习模型
- 一层编码、解码的复杂映射能力弱，难以学习复杂流形

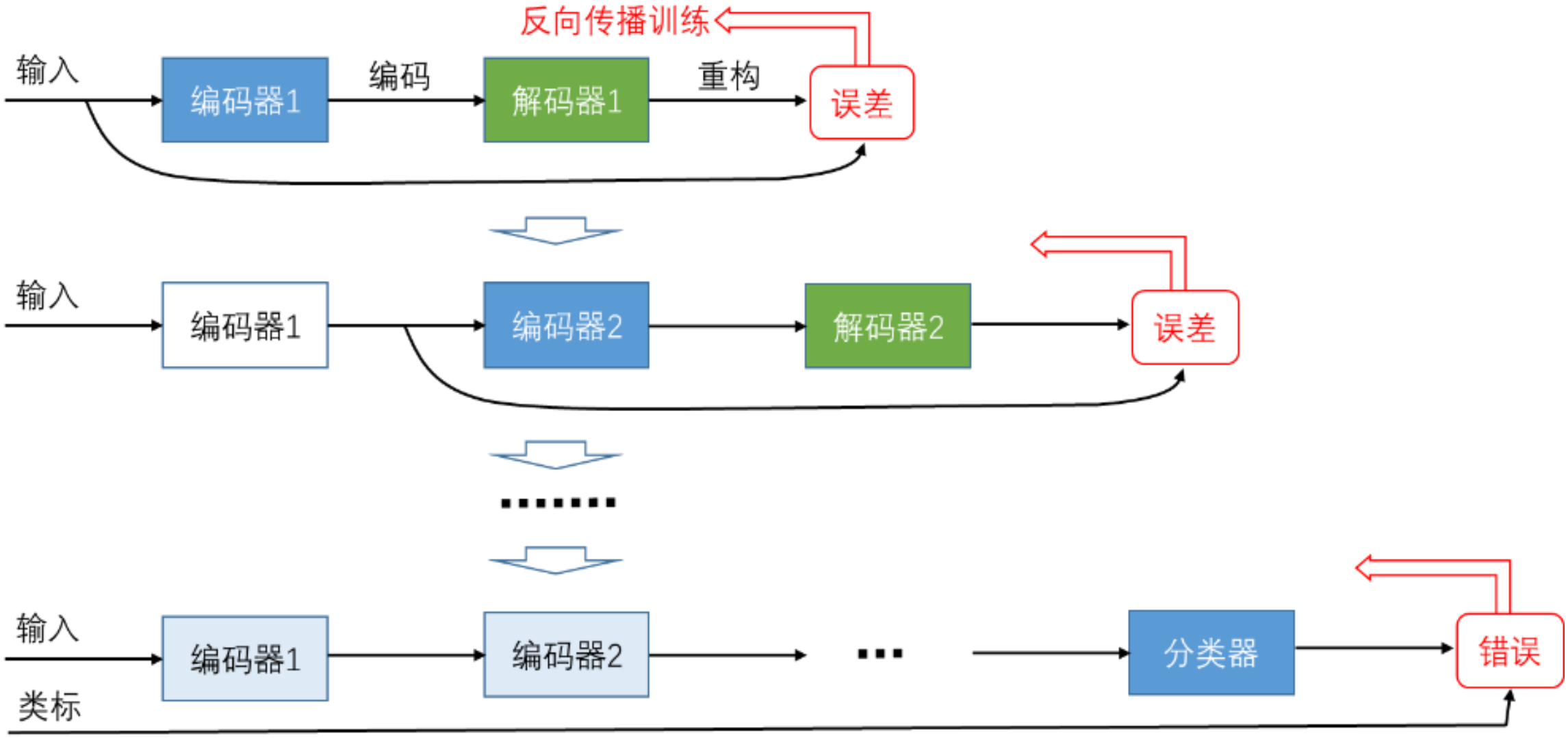


- 多层自编码器：
  - 增加多个隐层，用中间隐层（瓶颈层）节点输出作为对样本的低维表示即编码
- 其他变种：
  - 稀疏自编码器（Sparse AE）、去噪自编码器（Denoising AE）、收缩自编码器（Contractive AE） 等

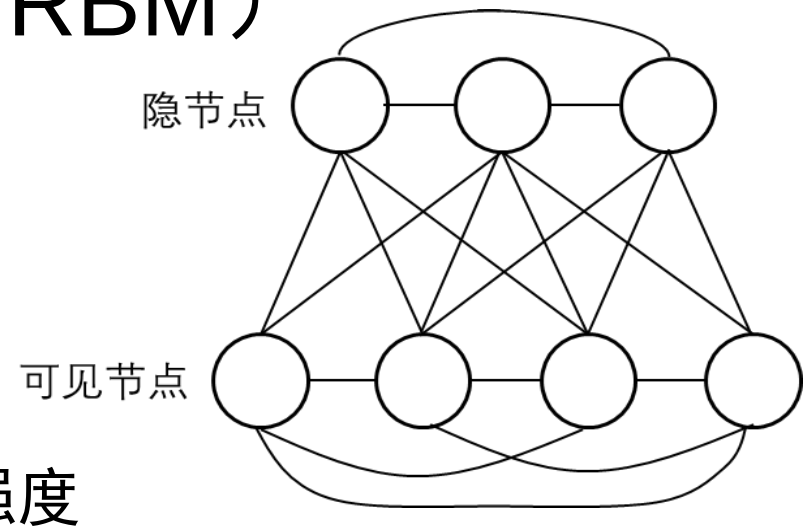


12.6.2 用多层自编码器构造深度神经网络

- **表示学习：**通过神经网络学习得到对样本更有效的特征表示
- 逐层训练全连接深度神经网络
  - 先构造并训练一层自编码器
  - 去掉解码器层，再接一个新的简单编码器进行训练，训练完成后再把解码器层去掉
  - 以此类推，一步步增加新的编码器层，然后进行误差反向传播训练。
- 逐级训练也称作多层网络的预训练，最后的训练过程也被称作对网络的微调


反向传播训练
输入
编码器1
编码
解码器1
重构
误差
输入
编码器1
编码器2
解码器2
误差
.....
输入
编码器1
编码器2
...
分类器
错误
类标

12.6.3 限制性玻尔兹曼机（RBM）



- 玻尔兹曼机（Boltmann Machines）
  - 可见节点$\boldsymbol{x}$：样本的特征变量，二值特征
  - 隐节点$\boldsymbol{h}$
  - 可见节点和隐节点之间全连接，边权值代表连接强度
- $E(\boldsymbol{x})$表示可见节点为$\boldsymbol{x}$时的能量，概率$p(\boldsymbol{x})$与能量的关系表示为：

$$p(\boldsymbol{x}) = \frac{e^{-E(\boldsymbol{x})}}{Z}$$

- 归一化因子Z：

$$Z = \sum_{\boldsymbol{x}} e^{-E(\boldsymbol{x})}$$

- 能量$E(\boldsymbol{x})$ 需考虑所有可能隐节点向量取值：

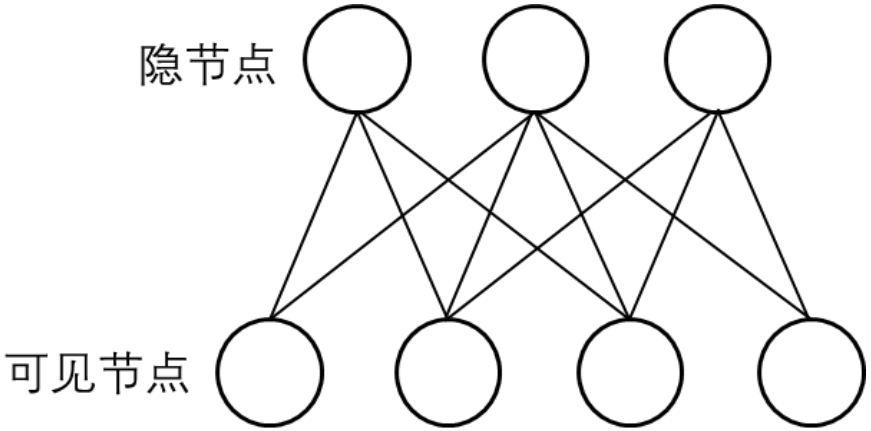
$$E(\boldsymbol{x}) = -\log \sum_{\boldsymbol{h}} e^{-E(\boldsymbol{x},\boldsymbol{h})}$$

- 对可见节点向量$\boldsymbol{x}$和隐节点向量$\boldsymbol{h}$，能量为：

$$E(\boldsymbol{x},\boldsymbol{h}) = -\boldsymbol{c}^T\boldsymbol{x} - \boldsymbol{b}^T\boldsymbol{h} - \boldsymbol{h}^T\boldsymbol{W}\boldsymbol{x} - \boldsymbol{x}^T\boldsymbol{U}\boldsymbol{x} - \boldsymbol{h}^T\boldsymbol{V}\boldsymbol{h}$$

  - $\boldsymbol{b}$和$\boldsymbol{c}$分别为隐节点和可见节点的偏置参数
  - $\boldsymbol{W}$：可见节点和隐节点之间的连接权值矩阵
  - $\boldsymbol{U}$和$\boldsymbol{V}$分别是可见节点和隐节点各自内部的连接权值矩阵
- 参数估计比较困难

- 限制性玻尔兹曼机（RBM）
  - 连接限制在可见层和隐层之间，同层直接无连接
  - 可见节点$\boldsymbol{x} \in \{0,1\}^D$，隐层节点$\boldsymbol{h} \in \{0,1\}^H$



- 联合概率为

$$p(\boldsymbol{x}, \boldsymbol{h}) = e^{-E(\boldsymbol{x},\boldsymbol{h})}/Z$$

- 其中归一化因子Z

$$Z = \sum_{\boldsymbol{x},\boldsymbol{h}} e^{-E(\boldsymbol{x},\boldsymbol{h})}$$

- 能量简化为：

$$E(\boldsymbol{x}, \boldsymbol{h}) = -\boldsymbol{c}^T\boldsymbol{x} - \boldsymbol{b}^T\boldsymbol{h} - \boldsymbol{h}^T\boldsymbol{W}\boldsymbol{x} = -\sum_j \sum_k W_{j,k} h_j x_k - \sum_k c_k x_k - \sum_j b_j h_j$$

- 限制性玻尔兹曼机（RBM）是一种生成模型，输入节点和隐节点的联合概率密度函数可写为

$$p(\boldsymbol{x},\boldsymbol{h}) = \frac{e^{-E(\boldsymbol{x},\boldsymbol{h})}}{Z}$$

$$= \frac{1}{Z}\exp\left(-\sum_j\sum_k \boldsymbol{W}_{j,k}h_j x_k - \sum_k c_k x_k - \sum_j b_j h_j\right)$$

$$= \frac{1}{Z}\prod_j\prod_k \exp(\boldsymbol{W}_{j,k}h_j x_k)\prod_k \exp(c_k x_k)\prod_j \exp(b_j h_j)$$

其中h_j和x_k分别表示隐层第j个节点和输入层第k个节点

- 早期也被称作“专家乘积”模型

- 根据联合分布，可以写出给定$\boldsymbol{x}$下$\boldsymbol{h}$的条件概率

$$p(\boldsymbol{h}|\boldsymbol{x}) = \frac{p(\boldsymbol{x},\boldsymbol{h})}{p(\boldsymbol{x})} = \frac{1}{Z'}\prod_j\prod_k \exp\left(\boldsymbol{W}_{j,k}h_j x_k\right)\prod_j \exp\left(b_j h_j\right) = \frac{1}{Z'}\prod_j \exp(b_j h_j + h_j \boldsymbol{W}_j.\boldsymbol{x})$$

- 由于$\boldsymbol{h}$为二值节点，可对隐节点概率归一化：

$$\boldsymbol{P}\left(h_j = 1\middle|\boldsymbol{x}\right) = \frac{P\left(h_j = 1\middle|\boldsymbol{x}\right)}{P\left(h_j = 0\middle|\boldsymbol{x}\right) + P\left(h_j = 1\middle|\boldsymbol{x}\right)} = \frac{\exp(b_j + \boldsymbol{W}_j.\boldsymbol{x})}{\exp(0) + \exp(b_j + \boldsymbol{W}_j.\boldsymbol{x})} = \frac{\exp(b_j + \boldsymbol{W}_j.\boldsymbol{x})}{1 + \exp(b_J + \boldsymbol{W}_j.\boldsymbol{x})}$$

即

$$P\left(h_j = 1\middle|\boldsymbol{x}\right) = \mathrm{sigm}(b_j + \boldsymbol{W}_j.\boldsymbol{x})$$

同理：

$$P\left(h_j = 0\middle|\boldsymbol{x}\right) = 1 - \mathrm{sigm}\left(b_j + \boldsymbol{W}_j.\boldsymbol{x}\right) = \mathrm{sigm}(-b_j - \boldsymbol{W}_j.\boldsymbol{x})$$

- 合并到统一公式：

$$p(\boldsymbol{h}|\boldsymbol{x}) = \prod_j p(h_j|\boldsymbol{x}) = \prod_j \mathrm{sigm}\big((2h_j - 1)(b_j + \boldsymbol{W}_{j.}\boldsymbol{x})\big)$$

- 类似的，我们也可以得到给定$\boldsymbol{h}$下$\boldsymbol{x}$的条件概率：

$$p(\boldsymbol{x}|\boldsymbol{h}) = \prod_k p(x_k|\boldsymbol{h}) = \prod_k \mathrm{sigm}((2x_k - 1)(c_k + \boldsymbol{h}\boldsymbol{W}_{.k}))$$

- 用样本进行RBM参数学习
- 对所有可能隐节点状态求$\boldsymbol{x}$的边缘分布：

$$P(\boldsymbol{x};\boldsymbol{\theta}) = \sum_{\boldsymbol{h}} p(\boldsymbol{x},\boldsymbol{h}) = \sum_{\boldsymbol{h}} e^{-E(\boldsymbol{x},\boldsymbol{h})}/Z$$

- 利用迭代求解最优参数：

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{\operatorname{argmax}}\, P(\boldsymbol{x};\boldsymbol{\theta}) = \underset{\boldsymbol{\theta}}{\operatorname{argmin}}(-\log P(\boldsymbol{x}))$$

- 梯度为：

$$\frac{\partial(-\log P(\boldsymbol{x}))}{\partial\boldsymbol{\theta}} = \frac{\partial\left(-\log\sum_{\boldsymbol{h}} e^{-E(\boldsymbol{x},\boldsymbol{h})} + \log\sum_{\boldsymbol{x},\boldsymbol{h}} e^{-E(\boldsymbol{x},\boldsymbol{h})}\right)}{\partial\boldsymbol{\theta}}$$

$$= \frac{1}{\sum_{\boldsymbol{h}} e^{-E(\boldsymbol{x},\boldsymbol{h})}} \sum_{h} e^{-E(\boldsymbol{x},\boldsymbol{h})} \frac{\partial E(\boldsymbol{x},\boldsymbol{h})}{\partial\boldsymbol{\theta}} - \frac{1}{\sum_{\boldsymbol{x},\boldsymbol{h}} e^{-E(\boldsymbol{x},\boldsymbol{h})}} \sum_{x,h} e^{-E(\boldsymbol{x},\boldsymbol{h})} \frac{\partial E(\boldsymbol{x},\boldsymbol{h})}{\partial\boldsymbol{\theta}}$$

$$= \sum_{\boldsymbol{h}} p(\boldsymbol{h}|\boldsymbol{x}) \frac{\partial E(\boldsymbol{x},\boldsymbol{h})}{\partial\boldsymbol{\theta}} - \sum_{\boldsymbol{x},\boldsymbol{h}} p(\boldsymbol{x},\boldsymbol{h}) \frac{\partial E(\boldsymbol{x},\boldsymbol{h})}{\partial\boldsymbol{\theta}}$$

- 梯度也可以写为：

$$\frac{\partial(-\log P(\boldsymbol{x}))}{\partial \boldsymbol{\theta}} = \mathbb{E}_{P(\boldsymbol{h}|\boldsymbol{x})} \frac{\partial E(\boldsymbol{x}, \boldsymbol{h})}{\partial \boldsymbol{\theta}} - \mathbb{E}_{P(\boldsymbol{x},\boldsymbol{h})} \frac{\partial E(\boldsymbol{x}, \boldsymbol{h})}{\partial \boldsymbol{\theta}}$$

  - 右边第一项是能量梯度在训练样本对应状态上的期望，沿梯度负方向模型对训练样本的似然度增大
  - 右边第二项是能量梯度对整个问题空间中的期望，沿梯度反方向调整参数模型对空间中任意样本的似然度都最大
  - 第二项在总梯度中是负项，因此使模型减小对空间中任意样本的似然度

- 各参数更新公式：
  - $\boldsymbol{W}$：

$$\frac{\partial(-\log P(\boldsymbol{x}))}{\partial \boldsymbol{W}} = -\mathbb{E}_{\boldsymbol{h}}(\boldsymbol{h}(\boldsymbol{x}(t))^T \boldsymbol{x}(t)) + \mathbb{E}_{\boldsymbol{x},\boldsymbol{h}} \boldsymbol{h}^T \boldsymbol{x}$$

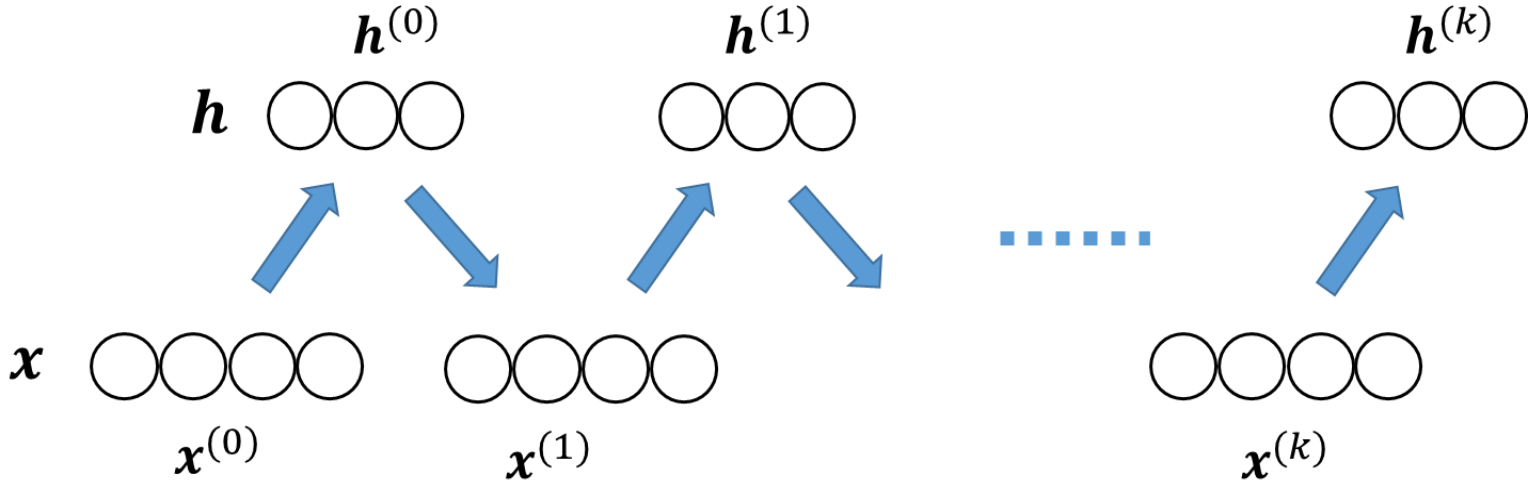
  - $\boldsymbol{c}$：

$$\frac{\partial(-\log P(\boldsymbol{x}))}{\partial \boldsymbol{c}} = -\boldsymbol{x}(t) + \mathbb{E}_{\boldsymbol{x},\boldsymbol{h}}$$

  - $\boldsymbol{b}$：

$$\frac{\partial(-\log P(\boldsymbol{x}))}{\partial \boldsymbol{b}} = -\mathbb{E}_{\boldsymbol{h}} \boldsymbol{h}(\boldsymbol{x}(t)) + \mathbb{E}_{\boldsymbol{x},\boldsymbol{h}} \boldsymbol{h}$$

- 迭代中须求两个期望值， $\mathbb{E}_{\boldsymbol{h}}(\,)$ 可根据$p(\boldsymbol{h}|\boldsymbol{x})$ 求得，如何求$\mathbb{E}_{\boldsymbol{x},\boldsymbol{h}}(\,)$?
- RBM训练的CD算法
  - 用蒙特卡洛迭代的方式来对状态空间进行采样
  - k步交替采样后得到的可见节点和隐节点状态$(\tilde{\boldsymbol{x}},\tilde{\boldsymbol{h}})$上的值作为对期望$\mathbb{E}_{\boldsymbol{x},\boldsymbol{h}}(\,)$的估计
- 在RBM网络学习中，只需k=1即可得到比较理想的效果



- 于是， t时刻各参数更新公式：
  - $\boldsymbol{W}$：

$$\boldsymbol{W}(t+1) = \boldsymbol{W}(t) + \eta\Delta\boldsymbol{W} = \boldsymbol{W}(t) + \eta\left(\boldsymbol{h}\left(\boldsymbol{x}(t)\right)^T\boldsymbol{x}(t) - \widetilde{\boldsymbol{h}}^T\widetilde{\boldsymbol{x}}\right)$$

  - $\boldsymbol{c}$：

$$\boldsymbol{c}(t+1) = \boldsymbol{c}(t) + \eta\Delta\boldsymbol{c} = \boldsymbol{c}(t) + \eta(\boldsymbol{x}(t) - \widetilde{\boldsymbol{x}})$$

  - $\boldsymbol{b}$：

$$\boldsymbol{b}(t+1) = \boldsymbol{b}(t) + \eta\Delta\boldsymbol{b} = \boldsymbol{b}(t) + \eta(\boldsymbol{h}(\boldsymbol{x}(t)) - \widetilde{\boldsymbol{h}})$$

【RBM学习算法：一步CD算法】

预设学习步长（学习率）为η

$\boldsymbol{W}$是RBM网络输入层与隐层之间的权值矩阵

$\boldsymbol{b}$是隐层偏置参数向量

$\boldsymbol{c}$是输入层偏置参数向量

（1）初始化

（2）加入下一个训练样本，记为$\boldsymbol{x_0}$

采样：

a）对所有隐节点，计算$P\left(h_j = 1 \middle| \boldsymbol{x_0}\right) = \mathrm{sigm}(b_j + \boldsymbol{W}_j.\boldsymbol{x_0})$ ，依此采样得向量$\boldsymbol{h_0}$

b）对所有可见节点，计算$P(x_k = 1|\boldsymbol{h}_0) = \mathrm{sigm}(c_k + \boldsymbol{h_0}\boldsymbol{W}_{.k})$，依此采样得向量$\boldsymbol{x_1}$

c）对所有隐节点，计算$P\left(h_j = 1 \middle| \boldsymbol{x_1}\right) = \mathrm{sigm}(b_j + \boldsymbol{W}_j.\boldsymbol{x_1})$ ，依此采样得向量$\boldsymbol{h_1}$

更新：

$$\boldsymbol{W} \Leftarrow \boldsymbol{W} + \eta\left(\boldsymbol{h}_0^T\boldsymbol{x}_0 - \boldsymbol{h}_1^T\boldsymbol{x}_1\right)$$

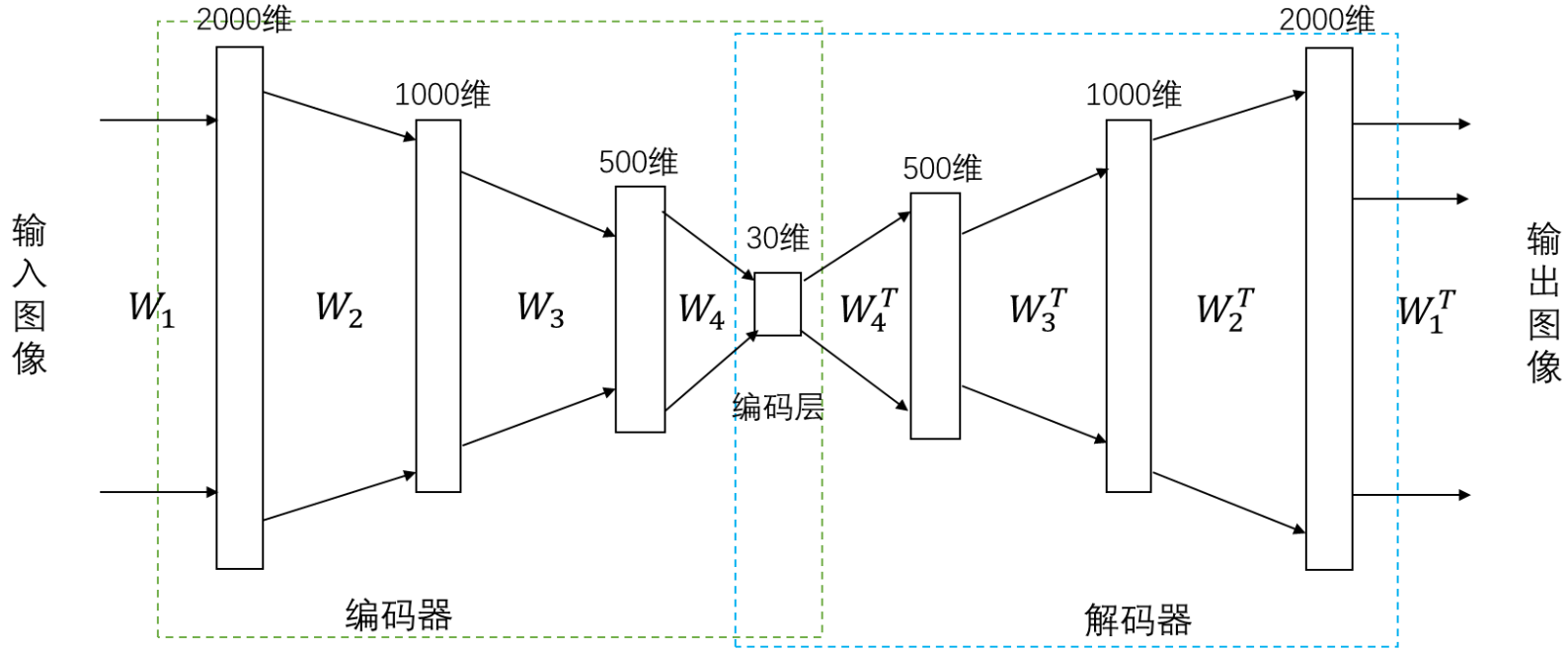
$$\boldsymbol{c} \Leftarrow \boldsymbol{c} + \eta(\boldsymbol{x}_0 - \boldsymbol{x}_1)$$

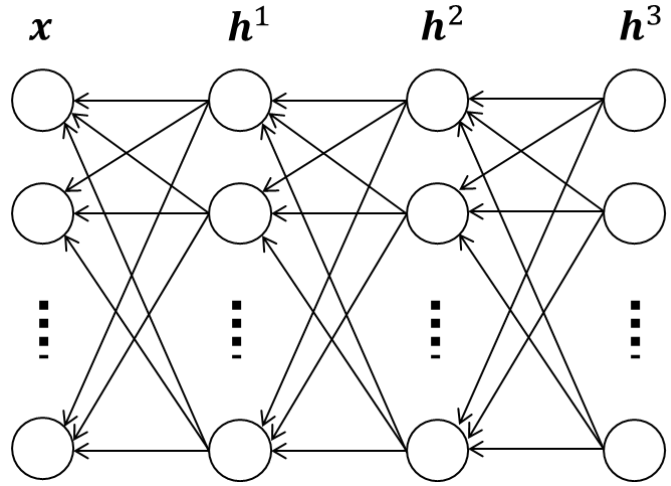
$$\boldsymbol{b} \Leftarrow \boldsymbol{b} + \eta(\boldsymbol{h}_0 - \boldsymbol{h}_1)$$

（3）检查收敛条件，如已收敛或已达预设训练次数则停止，否则转（2）

12.6.4 深度自编码器与深度信念网络（DBN）

- 相似点：同样需要逐层训练
- 区别：DBN利用RBM对每一层编码层进行预训练
  - 首先把输入节点和第一编码层节点看作RBM网络，利用CD算法训练
  - 再将第一，二编码层节点看作RBM网络， 利用CD算法训练，以此类推
  - “unrolling”展开：各编码层权值矩阵对称到解码器





- Sigmoid信念网络：类似多层感知器的贝叶斯网络
- 相邻两层条件概率由sigmoid函数刻画：

$$P(x_i = 1 \mid \boldsymbol{h}^1) = \mathrm{sigm}\left(b_i^0 + \sum_j W_{i,j}^1 h_j^1\right)$$

$$P\left(h_i^k = 1 \mid \boldsymbol{h}^{k+1}\right) = \mathrm{sigm}\left(b_i^k + \sum_j W_{i,j}^{k+1} h_j^{k+1}\right), \qquad k = 1, \cdots, l-1$$

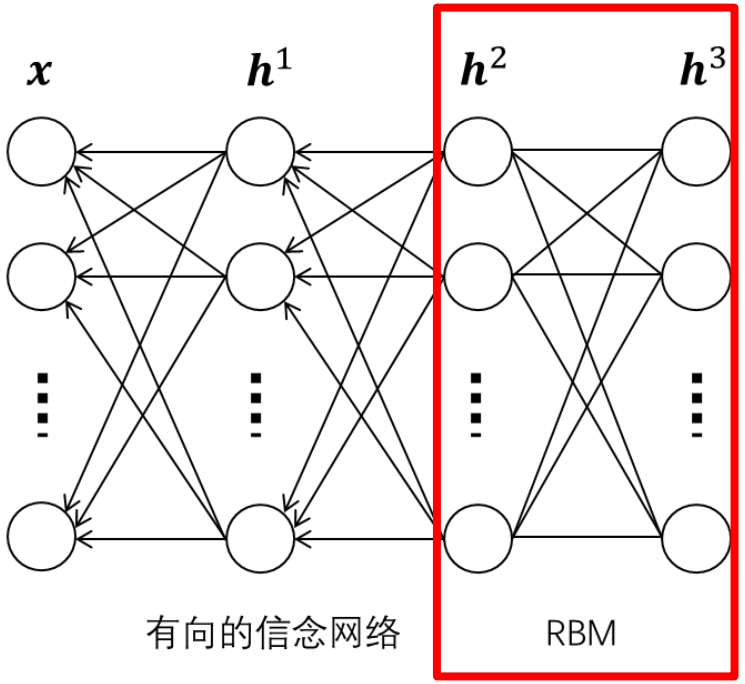
- 联合概率为

$$P\left(\boldsymbol{x}, \boldsymbol{h}^1, \cdots, \boldsymbol{h}^l\right) = P(\boldsymbol{h}^l)\left(\prod_{k=1}^{l-1} P\left(\boldsymbol{h}^k \mid \boldsymbol{h}^{k+1}\right)\right) P(\boldsymbol{x} \mid \boldsymbol{h}^1)$$

- 存在“解释排除”现象，难以进行模型推断

- 深度信念网络（Deep Belief Network，DBN）
  - 最后的隐层关系用无向RBM替代原模型，便于求解
  - 最后两层RBM网络联合概率

$$P(h^{l-1}, h^l) \propto exp\left(c^T h^{l-1} + b^T h^l + h^{l^T} W h^{l-1}\right)$$



- 整体概率分布：

$$P\left(\boldsymbol{x}, \boldsymbol{h}^1, \cdots, \boldsymbol{h}^l\right) = P(\boldsymbol{h}^{l-1}, \boldsymbol{h}^l)\left(\prod_{k=1}^{l-1} P\left(\boldsymbol{h}^k \middle| \boldsymbol{h}^{k+1}\right)\right) P(\boldsymbol{x}|\boldsymbol{h}^1)$$

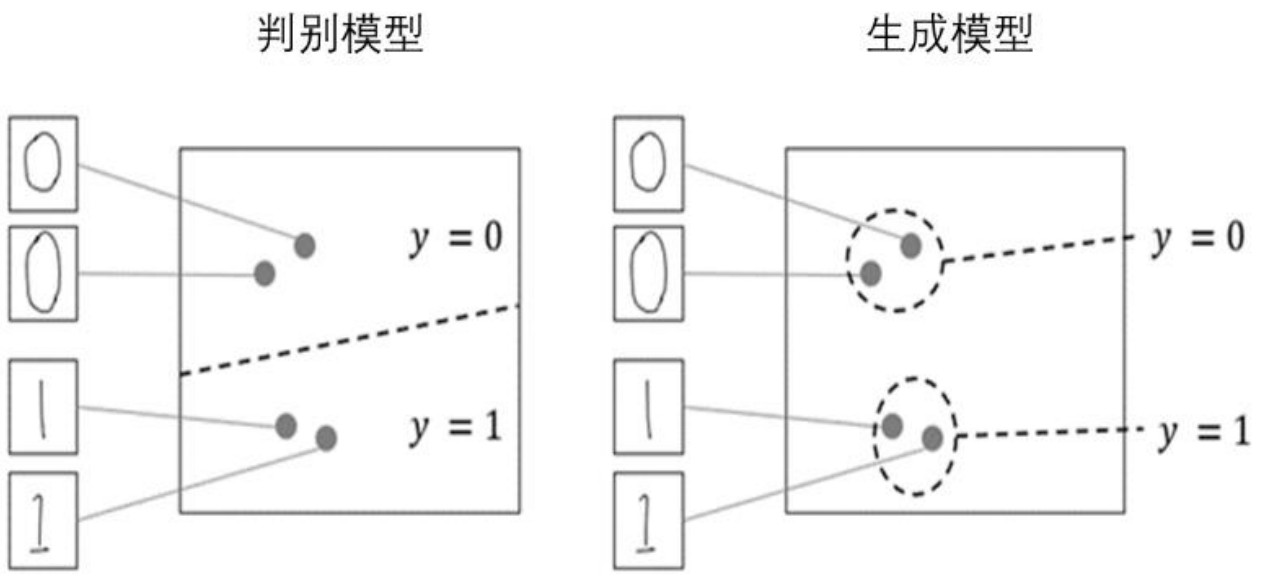
- 训练方法：
  - 逐层构建的贪婪学习算法，每步增加DBN模型对训练数据的对数似然函数的变分下界

- DBN属于生成模型，可以用来生成新样本
- 更广阔应用是将最顶端节点作为对样本新的特征表示，进行后续的分类或预测任务
  - 把DBN的训练看作是对多层感知器的非监督预训练
  - 再用监督学习的分类或预测目标去对网络进行再训练
- DBN是第一个非卷积的深度神经网络模型，也是人们接受并广泛使用“深度学习”这个概念的开始

- 深度信念网络
  - 专指DBN类似的模型，最后一层是无向的限制性玻尔兹曼机网络，而之前各层均为有向的Sigmoid信念网络
  - 是贝叶斯网络与人工神经网络的结合
- 深度神经网络（Deep Neural Networks，DNN）
  - 广泛概念，多层（通常指超过四层）结构、每层包含多个非线性计算单元、各层单元之间具有需通过训练确定的连接权值等参数的计算模型
  - 与之相关的机器学习方法统称为深度学习（Deep Learning）

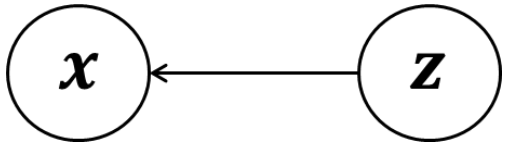
12.7 生成模型

- 生成式模型（generative model）：估计或模拟样本的概率分布
  - 对于分类问题，计算样本$\boldsymbol{x}$以及标签y联合概率分布$p(\boldsymbol{x}, y)$
- 判别式模型（discriminative model）
  - 对后验概率$P(y|\boldsymbol{x})$直接建模



12.7.1 变分自编码器（VAE）

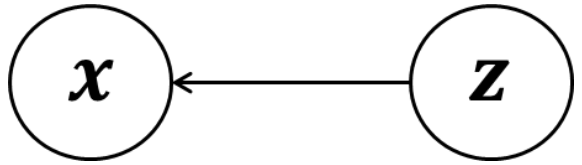
- 通过训练样本学习隐节点变量概率分布
- 看似与自编码器有相同构造，数学模型完全不同，本质上是概率图模型
- 模型的参数推断采用变分推断（Variational Inference ）方法



- 概率图中的变分推断：对样本$\boldsymbol{x}$，从一组隐含向量$\boldsymbol{z}$中依概率产生

$$p(\boldsymbol{x},\boldsymbol{z}) = p(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})$$

- 目标：推断$\boldsymbol{z}$的后验概率密度$p(\boldsymbol{z}|\boldsymbol{x})$

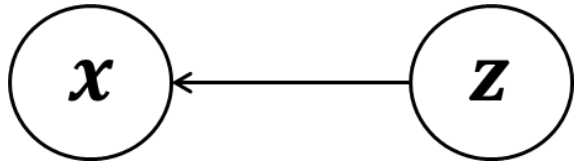


$$p(\boldsymbol{z}|\boldsymbol{x}) = \frac{p(\boldsymbol{x}|\boldsymbol{z})p(\boldsymbol{z})}{p(\boldsymbol{x})}$$

其中：

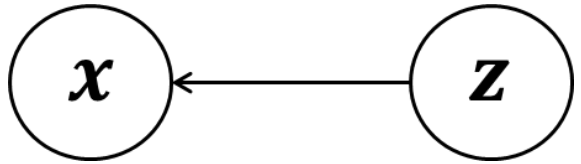
$$p(\boldsymbol{x}) = \int_{\boldsymbol{z}} p(\boldsymbol{z})p(\boldsymbol{x}|\boldsymbol{z})d\boldsymbol{z}$$

- $p(\boldsymbol{x})$是观测样本$\boldsymbol{x}$的边缘密度，也称作“证据”，但难以求解
- 可以利用马尔可夫链蒙特卡洛（MCMC）求解，但效率较低



- 变分推断：对函数和泛函的微小变化（称作变分）寻找泛函的极值
- 引入某个易计算的函数变分函数$q(\boldsymbol{z})$，尽可能接近$p(\boldsymbol{z}|\boldsymbol{x})$
- 如何定义“接近”？
- 衡量概率密度函数之间的相似度：KL散度
- 设x是一个离散的随机变量，它的概率分布函数是$P(x)$
- 定义信息量$I(x) = -\log P(x)$，则随机事件的熵：

$$H(x) = -\sum_{x} P(x)\log P(x)$$



- 同一随机变量，还存在另一个概率分布函数$Q(x)$
- $P(x)$和$Q(x)$的差异定义为它们熵的差，即KL散度：

$$D_{KL}(Q||P) = \sum_x Q(x)\log Q(x) - \sum_x Q(x)\log P(x) = \sum_x Q(x)\log\frac{Q(x)}{P(x)}$$

- 可等价写为：

$$D_{KL}(Q||P) = -\sum_x Q(x)\log\frac{P(x)}{Q(x)}$$

- 一般情况下，$D_{KL}(P||Q) \neq D_{KL}(Q||P)$
- 此外，还满足非负性：$D_{KL}(P||Q) \geq 0$

- 引入变分函数$q(\boldsymbol{z})$来逼近难求解的后验概率密度函数$p(\boldsymbol{z}|\boldsymbol{x})$
- KL散度为：

$$D_{KL}(q(\boldsymbol{z})||p(\boldsymbol{z}|\boldsymbol{x})) = \int_{\boldsymbol{z}} q(\boldsymbol{z})\log\frac{q(\boldsymbol{z})}{p(\boldsymbol{z}|\boldsymbol{x})}d\boldsymbol{z} = \int_{\boldsymbol{z}} q(\boldsymbol{z})\log\frac{q(\boldsymbol{z})p(\boldsymbol{x})}{p(\boldsymbol{x},\boldsymbol{z})}d\boldsymbol{z}$$

$$= \int_{\boldsymbol{z}} q(\boldsymbol{z})(\log q(\boldsymbol{z}) + \log p(\boldsymbol{x}) - p(\boldsymbol{x},\boldsymbol{z}))d\boldsymbol{z}$$

$$= \log p(\boldsymbol{x}) + \int_{\boldsymbol{z}} q(\boldsymbol{z})\log q(\boldsymbol{z})d\boldsymbol{z} - \int_{\boldsymbol{z}} q(\boldsymbol{z})\log p(\boldsymbol{x},\boldsymbol{z})d\boldsymbol{z}$$

- 由此可得：

$$\log p(\boldsymbol{x}) = D_{KL}(q(\boldsymbol{z})||p(\boldsymbol{z}|\boldsymbol{x})) - \int_{\boldsymbol{z}} q(\boldsymbol{z})\log q(\boldsymbol{z})d\boldsymbol{z} + \int_{\boldsymbol{z}} q(\boldsymbol{z})\log p(\boldsymbol{x},\boldsymbol{z})d\boldsymbol{z}$$

$$= D_{KL}(q(\boldsymbol{z})||p(\boldsymbol{z}|\boldsymbol{x})) + \int_{\boldsymbol{z}} q(\boldsymbol{z})\log\frac{p(\boldsymbol{x},\boldsymbol{z})}{q(\boldsymbol{z})}d\boldsymbol{z}$$

- 由非负性可知：

$$\log p(\boldsymbol{x}) \geq \int_{\boldsymbol{z}} q(\boldsymbol{z}) \log \frac{p(\boldsymbol{x}, \boldsymbol{z})}{q(\boldsymbol{z})} d\boldsymbol{z}$$

  $\log p(\boldsymbol{x})$为变分下界

- 令

$$\mathcal{L}(q) \triangleq \int_{\boldsymbol{z}} q(\boldsymbol{z}) \log \frac{p(\boldsymbol{x}, \boldsymbol{z})}{q(\boldsymbol{z})} d\boldsymbol{z}$$

- $\mathcal{L}(q)$称作证据下界，简写为ELBO
- 最大化证据下界ELBO即等价于最小化KL散度

- 分解$\mathcal{L}(q)$中联合概率有：

$$\mathcal{L}(q) = \int_{\boldsymbol{z}} q(\boldsymbol{z}) \log p(\boldsymbol{x}|\boldsymbol{z}) d\boldsymbol{z} + \int_{\boldsymbol{z}} q(\boldsymbol{z}) \log \frac{p(\boldsymbol{z})}{q(\boldsymbol{z})} d\boldsymbol{z}$$
$$= E_{\boldsymbol{z}\sim q(\boldsymbol{z})} \log p(\boldsymbol{x}|\boldsymbol{z}) - D_{KL}(q(\boldsymbol{z})||p(\boldsymbol{z}))$$

第一项为似然函数，第二项为KL散度

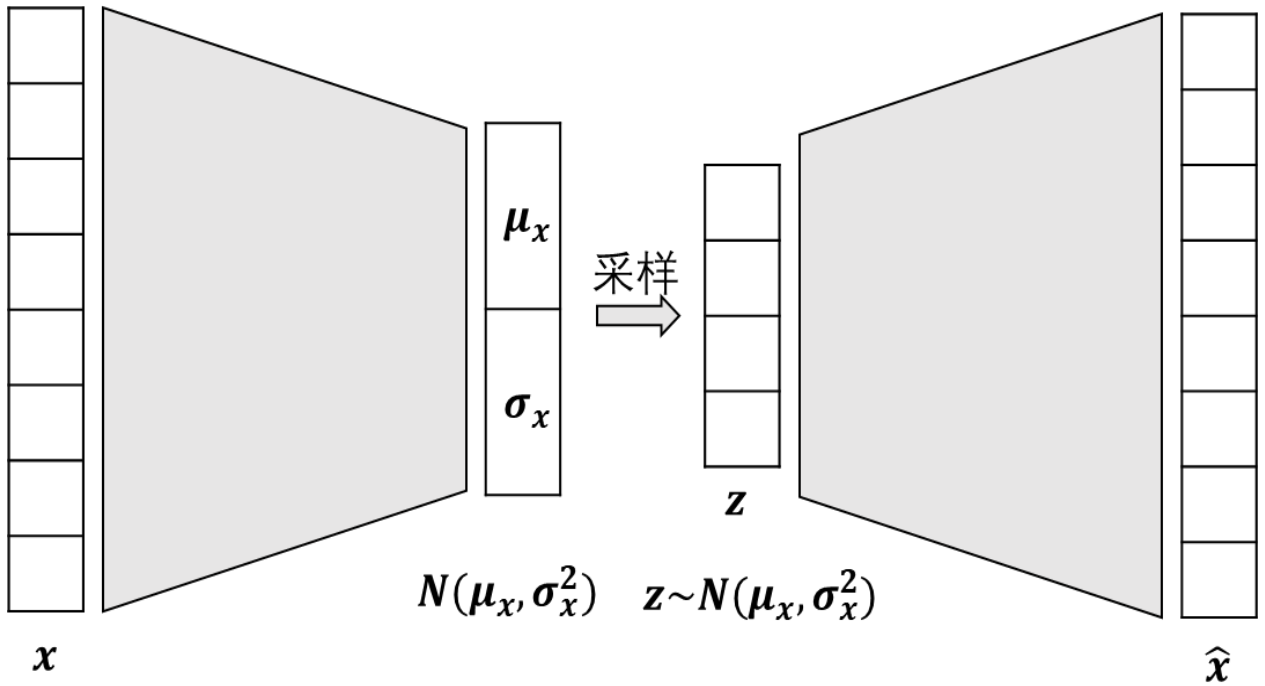
- 上式中$q(\boldsymbol{z})$可看做在观测样本下从给定函数族中求解的，因此可以看作$\boldsymbol{x}$到$\boldsymbol{z}$的条件密度$q(\boldsymbol{z}|\boldsymbol{x})$

- VAE中用编码层来实现$q(\boldsymbol{z}|\boldsymbol{x})$，用解码层来实现$p(\boldsymbol{x}|\boldsymbol{z})$
- 目标函数：最大化ELBO：

$$\max \ \mathcal{L}(q) = E_{\boldsymbol{z}\sim q(\boldsymbol{z})}\log p(\boldsymbol{x}|\boldsymbol{z}) - D_{KL}(q(\boldsymbol{z}|\boldsymbol{x})||p(\boldsymbol{z}))$$

- 指定隐层为各项独立的多元正态分布$\boldsymbol{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$，目标等价于：

$$\min \quad \|\boldsymbol{x} - \hat{\boldsymbol{x}}\|^2 + D_{KL}(q(\boldsymbol{z}|\boldsymbol{x})||N(\boldsymbol{\mu}, \boldsymbol{\sigma}^2))$$

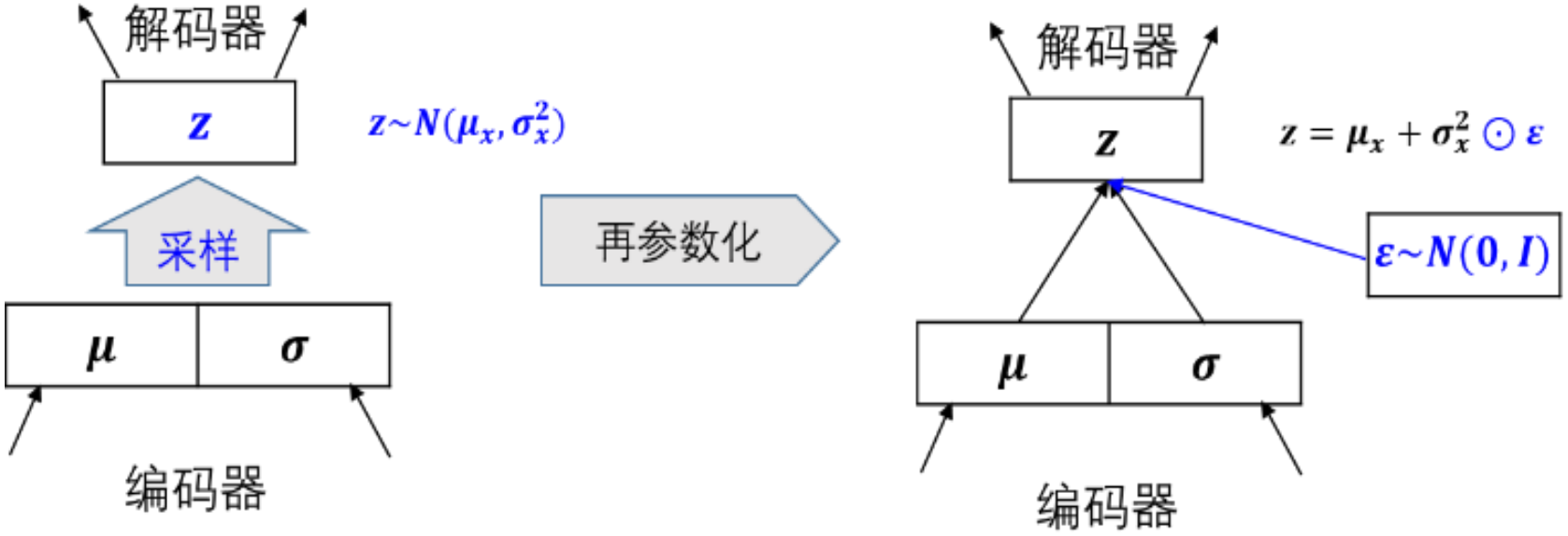


- 由于中间存在采样，梯度无法传播到编码层
- 解决办法：“再参数化技巧”
  - 原模型中解码层输入向量z通过采样得到：

$$z|x \sim N(\mu_x, \sigma_x^2)$$

  - 再参数化，系数ε保留了随机性，但不是训练的参数，使得梯度可以传播：

$$z|x = \mu_x + \sigma_x^2 \odot \varepsilon, \qquad \varepsilon \sim N(0, I)$$



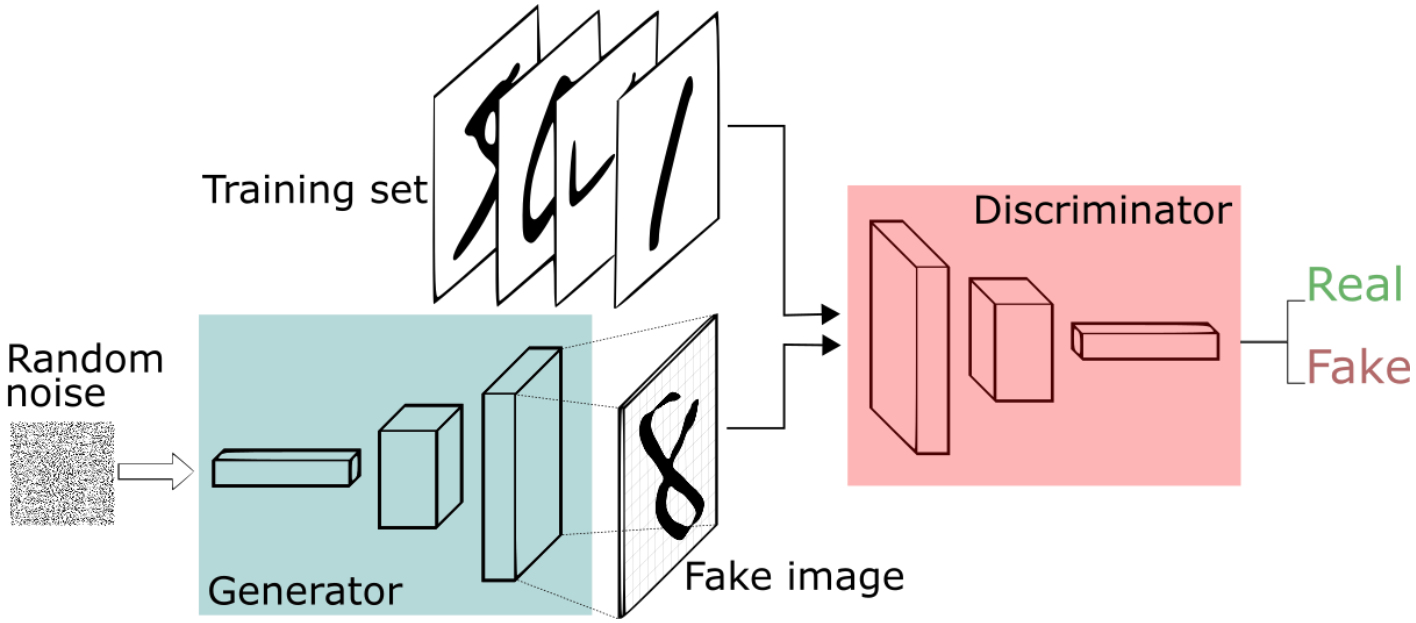
12.7.2 生成对抗网络（GAN）

GAN全称对抗生成网络，顾名思义是生成模型的一种，而他的训练则是处于一种对抗博弈状态中的。 GAN的主要结构包括一个**生成器**G（Generator）和一个**判别器**D（Discriminator）

以手写字为例：

假如拥有大量的手写数字的数据集，通过GAN生成一些能够以假乱真的手写字图片。主要由如下两个部分组成：

- 定义一个模型来作为生成器（图中蓝色部分Generator），能够输入一个向量，输出手写数字大小的像素图像。
- 定义一个分类器来作为判别器（图中红色部分Discriminator）用来判别图片来自数据集中的还是生成器中生成的，输入为手写图片，输出为判别图片的标签。



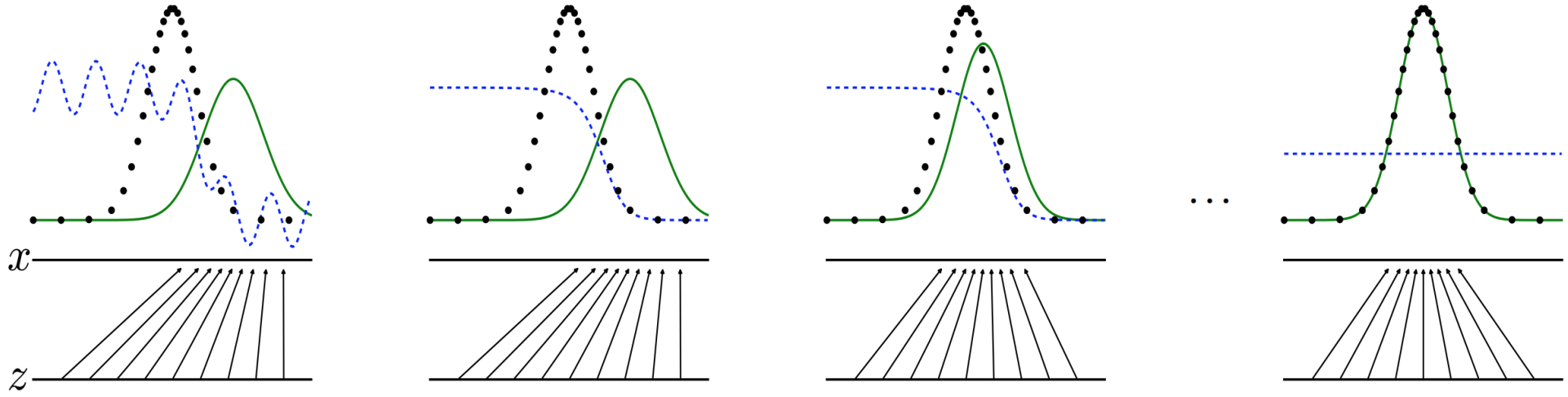
训练流程：

•初始化判别器D的参数 θ_d 和生成器G的参数 θ_g

•从真实样本中采样 m 个样本 $\{x_1, x_2, \ldots, x_m\}$，从先验分布噪声中采样 m 个噪声样本 $\{z_1, z_2, \ldots, z_m\}$并通过生成器获取 m 个生成样本 $\{\tilde{x}^1, \tilde{x}^2, \ldots, \tilde{x}^m\}$。固定生成器$G$，训练判别器$D$尽可能好地准确判别真实样本和生成样本，尽可能大地区分正确样本和生成的样本。

•循环k次更新判别器之后，使用较小的学习率来更新一次生成器的参数，训练生成器使其尽可能能够减小生成样本与真实样本之间的差距，也相当于尽量使得判别器判别错误。

•多次更新迭代之后，最终理想情况是使得判别器判别不出样本来自于生成器的输出还是真实的输出。亦即最终样本判别概率均为0.5。



(a) (b) (c) (d)

注：图中的**黑色虚线**表示真实的样本的分布情况，**蓝色虚线**表示判别器判别概率的分布情况，**绿色实线**表示生成样本的分布。 Z 表示噪声， Z 到 x 表示通过生成器之后的分布的映射情况。

目标是使用生成样本分布（绿色实线）去拟合真实的样本分布（黑色虚线），来达到生成以假乱真样本的目的。

- 在初始状态（**a**）时，生成器生成的分布和真实分布区别较大，并且判别器判别出样本的概率不是很稳定，因此会先训练判别器来更好地分辨样本。
- 通过多次训练判别器来达到（**b**）状态，判别样本区分效果显著，然后再对生成器进行训练。
- 训练生成器之后达到（**c**）样本状态，此时生成器分布相比之前，逼近了真实样本分布。
- 经过多次反复训练迭代之后，最终希望能够达到（**d**）状态，生成样本分布拟合于真实样本分布，并且判别器分辨不出样本是生成的还是真实的（判别概率均为0.5）

【GAN分批随机梯度下降训练算法】

对每一轮训练：{

（1）对判别器进行k步优化（其中k为需要设置的超参数），对其中每一步：

a）从生成器隐变量先验密度$p_g(\boldsymbol{z})$中采样生成一批m个生成样本$\{\boldsymbol{z}^{(1)},\cdots,\boldsymbol{z}^{(m)}\}$

b）从训练样本集中采样一批m个真实样本$\{\boldsymbol{x}^{(1)},\cdots,\boldsymbol{x}^{(m)}\}$

c）对判别器的参数θ_D求目标函数的梯度

$$\nabla_{\theta_D}\frac{1}{m}\sum_{i=1}^{m}\left(\log D\left(\boldsymbol{x}^{(i)}\right)+\log\left(1-D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right)\right)$$

用该梯度上升的方向更新判别器参数θ_D，即

$$\theta_D\leftarrow\theta_D+\eta\nabla_{\theta_D}\frac{1}{m}\sum_{i=1}^{m}\left(\log D\left(\boldsymbol{x}^{(i)}\right)+\log\left(1-D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right)\right)$$

其中η为步长即学习率。

（2）对生成器进行优化

a） 从生成器的隐变量概率密度$p_g(\boldsymbol{z})$中采样生成一批m个生成样本$\{\boldsymbol{z}^{(1)},\cdots,\boldsymbol{z}^{(m)}\}$

b） 对生成器参数θ_G求目标函数的梯度

$$\nabla_{\theta_G}\frac{1}{m}\sum_{i=1}^{m}\log\left(1-D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right)$$

用该梯度下降的方向更新生成器参数θ_G，即

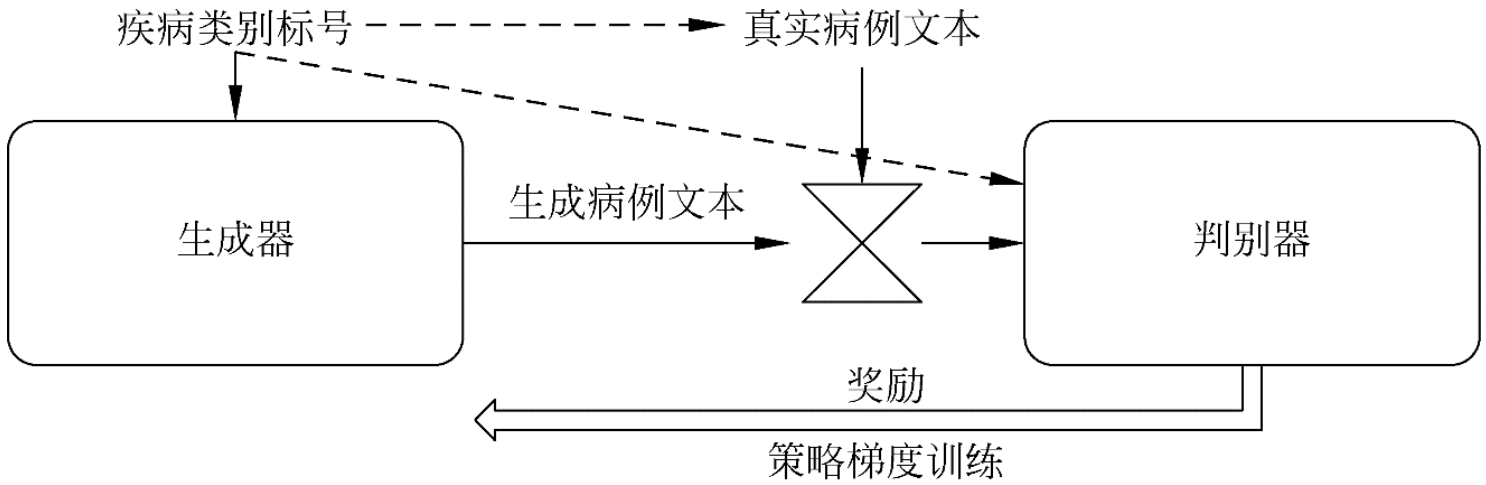
$$\theta_G\leftarrow\theta_G-\eta\nabla_{\theta_G}\frac{1}{m}\sum_{i=1}^{m}\log\left(1-D\left(G\left(\boldsymbol{z}^{(i)}\right)\right)\right)$$

} 如此往复迭代训练，直到达到预设训练次数。

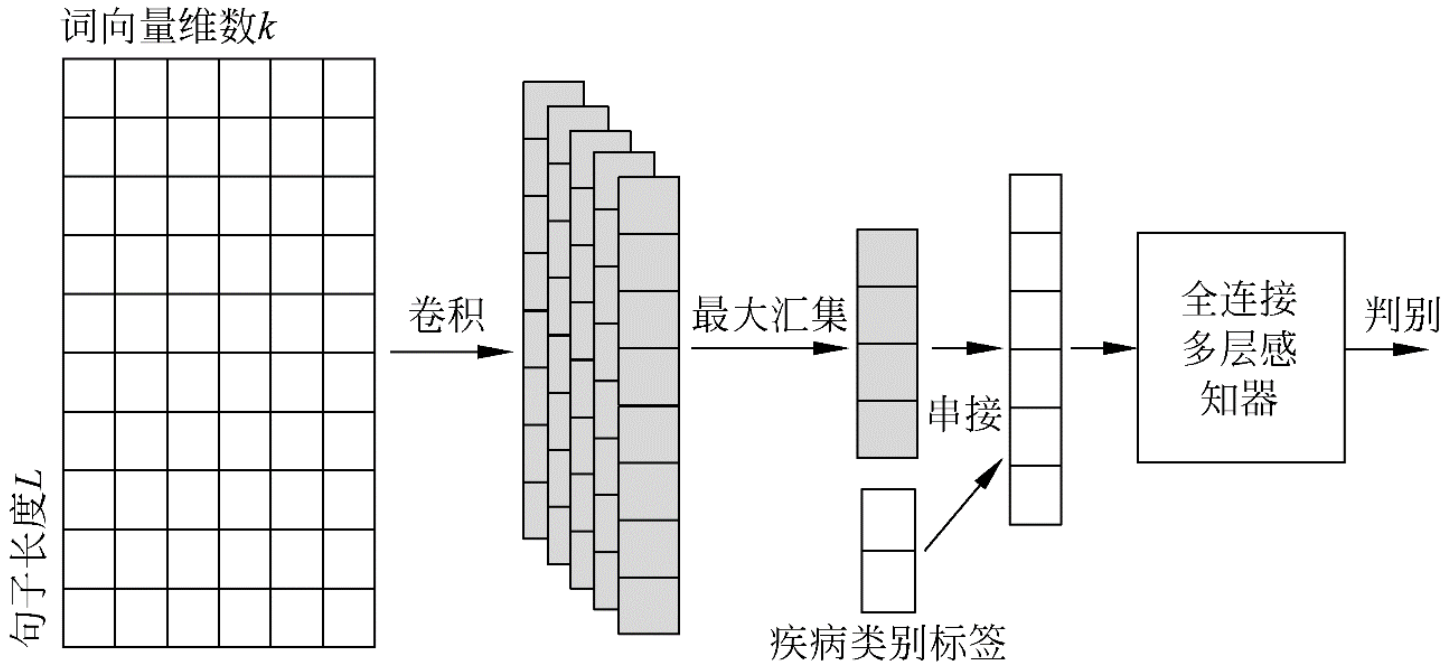
12.8 综合应用举例
12.8.1 中文病历文本生成

- 采用强化学习的生存对抗网络按病种产生仿真的中文病历
- 使用条件生成对抗网络引入疾病类型标签
- 使用word2vec方法将词编码为向量
- 使用长短时记忆（LSTM）神经网络作为生成器模型
- 使用卷积神经网络（CNN）作为判别器模型
- mtGAN: Medical Text Generative Adversarial Network

- mtGAN基本结构和判别器模型



(a)



(b)

- mtGAN与强化学习的对应关系
  - 策略：生成器G
  - 动作：下一步要产生的词（x_t）
  - 状态：当前已经生成的文本片段（$x_{1:t-1}$）
  - 奖励：对于由生成器G生成的句子，判别器判定它为真实样本的概率P
- 学习过程加速
  - 训练过程中常出现句子中个别片段不合理的情况
  - 采用蒙特卡洛搜索方法对句子部分片段进行多次采样
  - 避免对整个句子进行奖励或惩罚，定位错误位置，加快学习速度

- 生成结果示例
  - 肺炎：

    “患者于1周前无明显诱因出现咳嗽，咳白色粘痰，伴活动后加重，休息后可缓解，间断服用镇咳药物等治疗，未行正规诊治。”

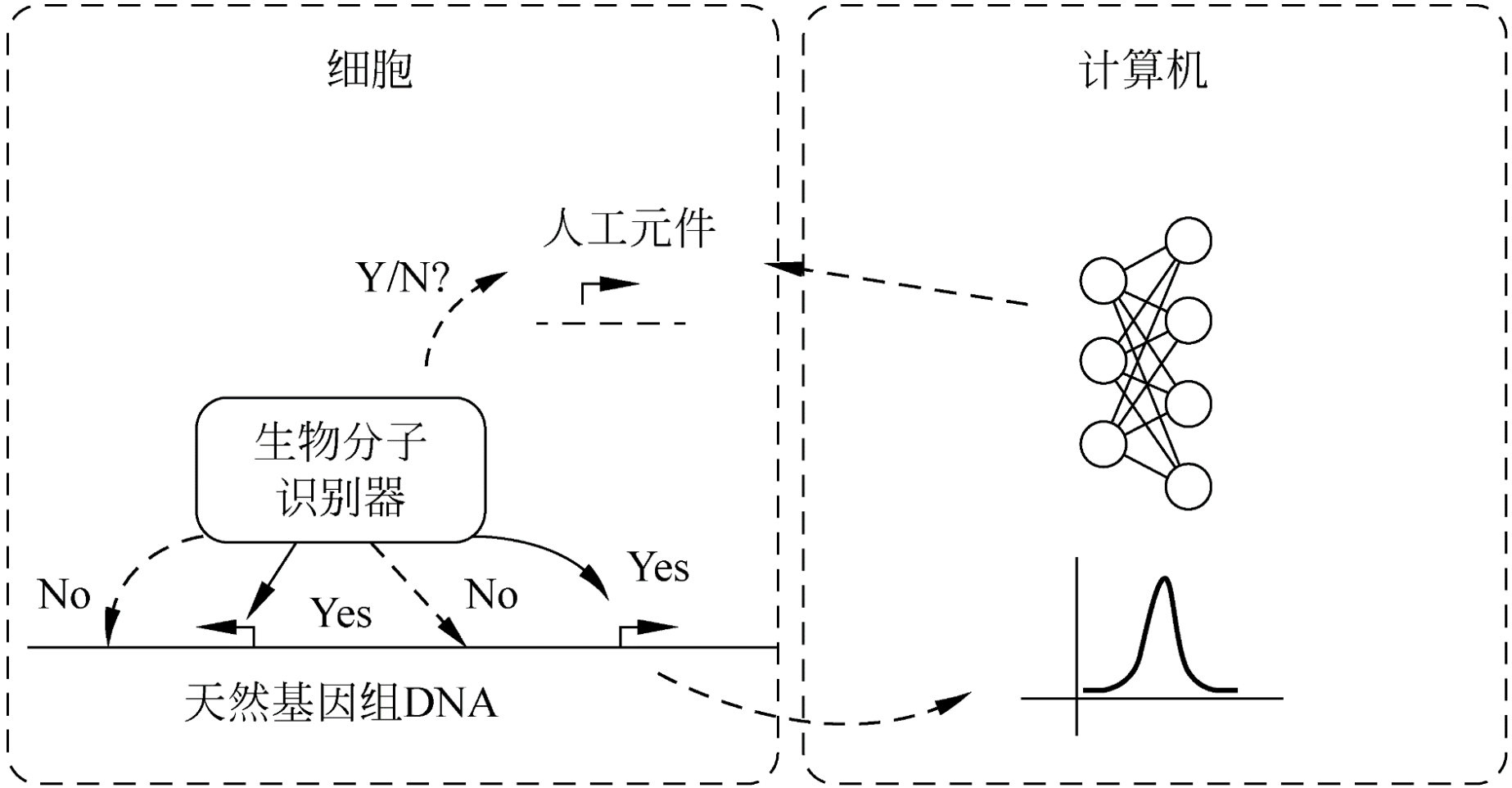
  - 肺癌：

    “患者10多年前开始出现咳嗽、咳痰，痰中带血，当地医院查胸部CT示纵膈肿大淋巴结，右肺下叶结节，后行气管镜时治疗收入院。”

12.8.2 人工基因调控元件的生成

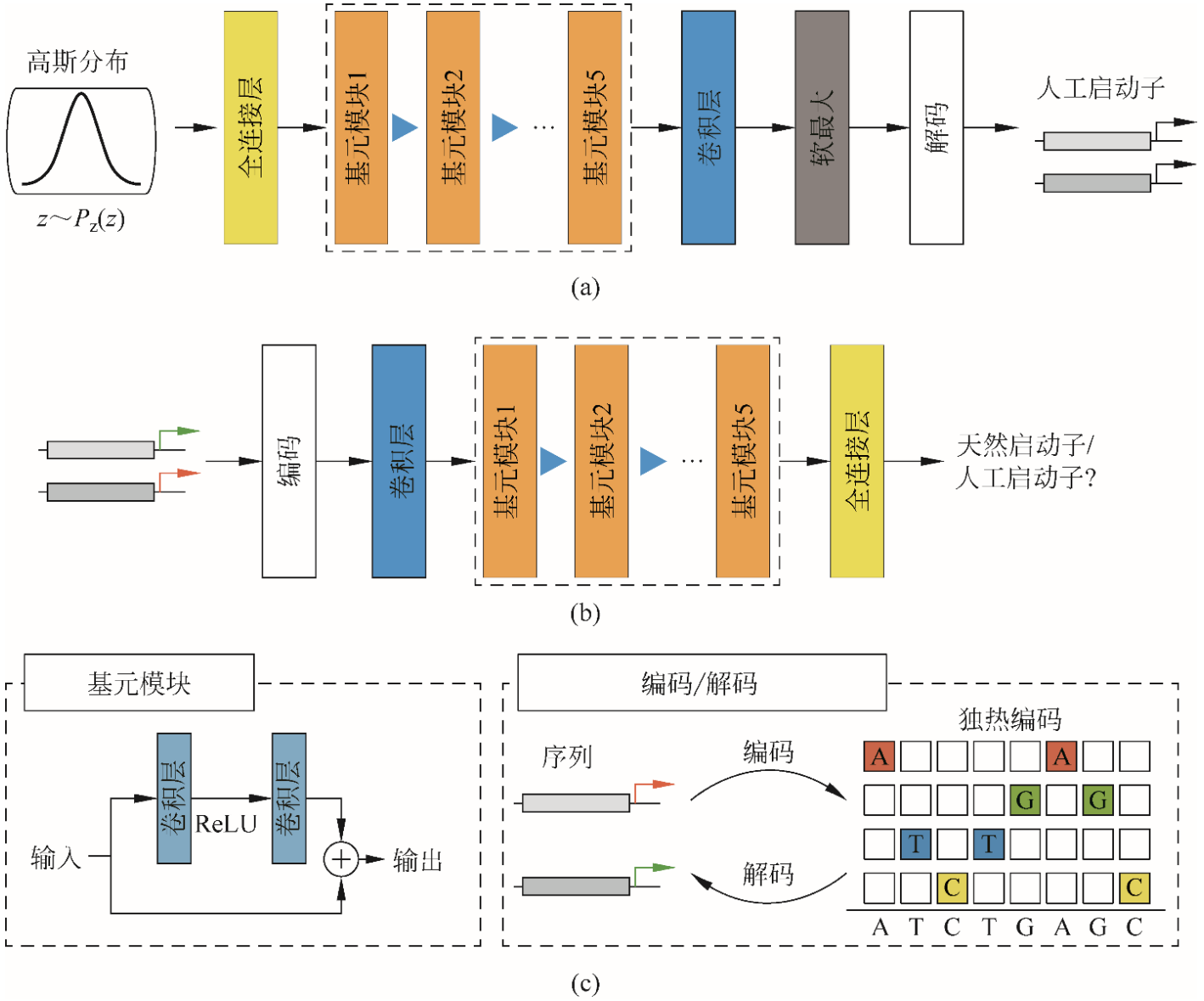
- 获得基因调控元件的方式
  - 自然进化：需要漫长的时间
  - 随机突变加筛选：成功率低，且难以发现全新的调控元件
- 从模式识别视角来思考这一问题
  - 人工设计的基因调控元件需要被识别为有功能的元件
  - 建模成一个分布估计和采样的问题：
    - 利用天然生物基因组中已知元件作为训练样本
    - 通过机器学习来学习这些样本的概率分布模型
    - 从学习到的分布中抽样产生新的、能够被细胞识别的人工基因调控元件
  - 利用生成对抗网络GAN来实现

- 基因调控元件的分布估计与生成的基本原理

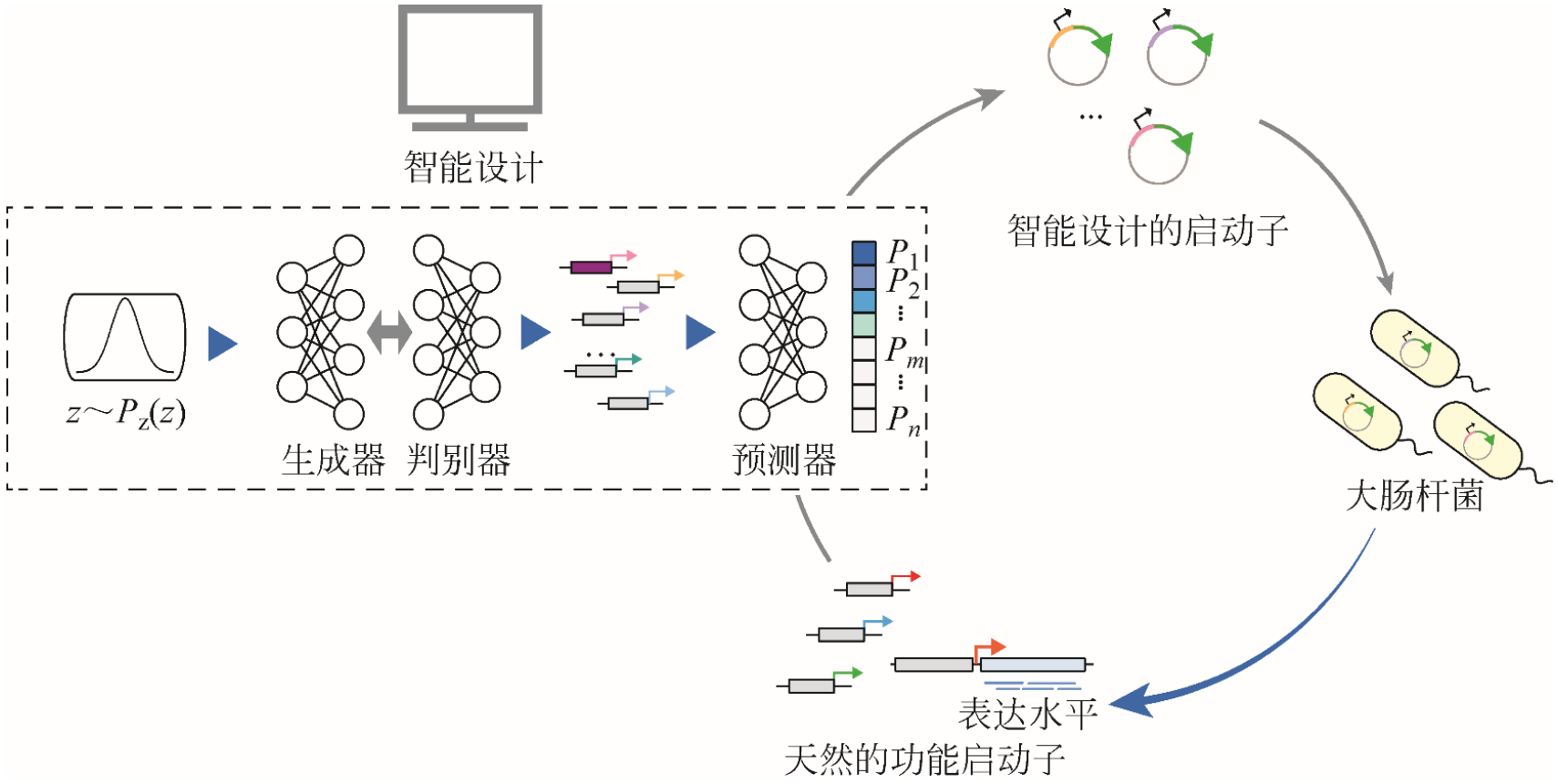


- 模型设计
  - 生成模型设计
    - 将低维隐变量送入生成器产生序列编码
    - 将天然序列送入模型
    - 判别器尽量区分生成器产生的序列与天然的序列
    - 生成器尽量产生更加接近天然序列的样本，来欺骗判别器
    - 分别对判别器和生成器进行梯度优化
  - 回归筛选模型设计
    - 输入：DNA序列
    - 输出：所调控的下游蛋白质的产出量
    - 筛选出潜在具有较高生物活性的样本，增加实验成功率

- 用生成对抗网络进行人工自动自生成的方法示意图



- 物理-虚拟-物理世界的循环映射过程
  - 将虚拟设计的样本合成为DNA，通过生物学装载到细胞内进行观测
  - 根据实验结果对模型进行循环优化



- 实验结果
  - 设计的大肠杆菌基因启动子有70%都具有生物活性
  - 部分元件的性能优于天然启动子
  - 与天然启动子相似性很低，降低了同源重组风险
  - 人工元件比天然序列的转录活性更高
- 推动工程生物系统更加高效、安全、可控的智能化设计和构建
- 极大促进代谢工程、分子育种、基因治疗等领域的发展

12.9 深度学习算法中的部分常用技巧

- 训练顺序
  - 梯度下降

$$\boldsymbol{w}(t+1) = \boldsymbol{w}(t) + (1-\alpha)\Delta\boldsymbol{w}_{bp}(t) + \alpha(\boldsymbol{w}(t) - \boldsymbol{w}(t-1))$$

  - 批量梯度下降
    - 每轮训练从训练集中选取一批样本计算梯度
    - 训练样本的批次划分要有随机性

- 学习率
  - 自适应梯度下降法
    - 让学习率在训练过程中进行动态变化
  - 二阶优化算法
    - 考虑目标函数的二阶导数信息
    - 经典算法包括牛顿法、共轭梯度法等
- 初始化
  - 在一个给定范围内对参数进行随机初始化
  - 条件允许时尝试不同的初始化参数

- 非线性激活函数
  - Sigmoid
    - 两侧的导数趋近于0，深度网络的梯度难以回传
    - 导致模型推广性较差
  - tanh
    - 随机梯度下降更趋近自然梯度，收敛速度更快
  - ReLU
    - 深度神经网络使用最广泛的激活函数
    - 缓解梯度消散，使神经网络具有稀疏表达的能力
    - 改进形式的ReLU函数使$s < 0$区域内也有梯度

- 随机舍弃
  - 舍弃法（dropout）
    - 每轮训练中，随机选择部分隐节点输出置零
    - 这些隐节点的参数也暂时不更新
    - 也属于一种正则化的方法
  - 舍弃的作用
    - 避免某些神经元共同激活，削弱神经元之间的关联适应性，增强推广能力
  - 实现细节
    - 相应地增加网络宽度
    - 训练时间上升到原来的2-3倍左右

- 归一化
  - 常用批量归一化（batch normalization）算法
  - 核心思想：在批量梯度下降中设法使每一批数据的分布相同
  - 对神经网络每层输入做下面的归一化运算

$$\hat{x}_i = \frac{x_i - \mu_x}{\sqrt{\sigma_x^2}}$$

  - 还需对特征做尺度调整和平移，提升特征表达能力

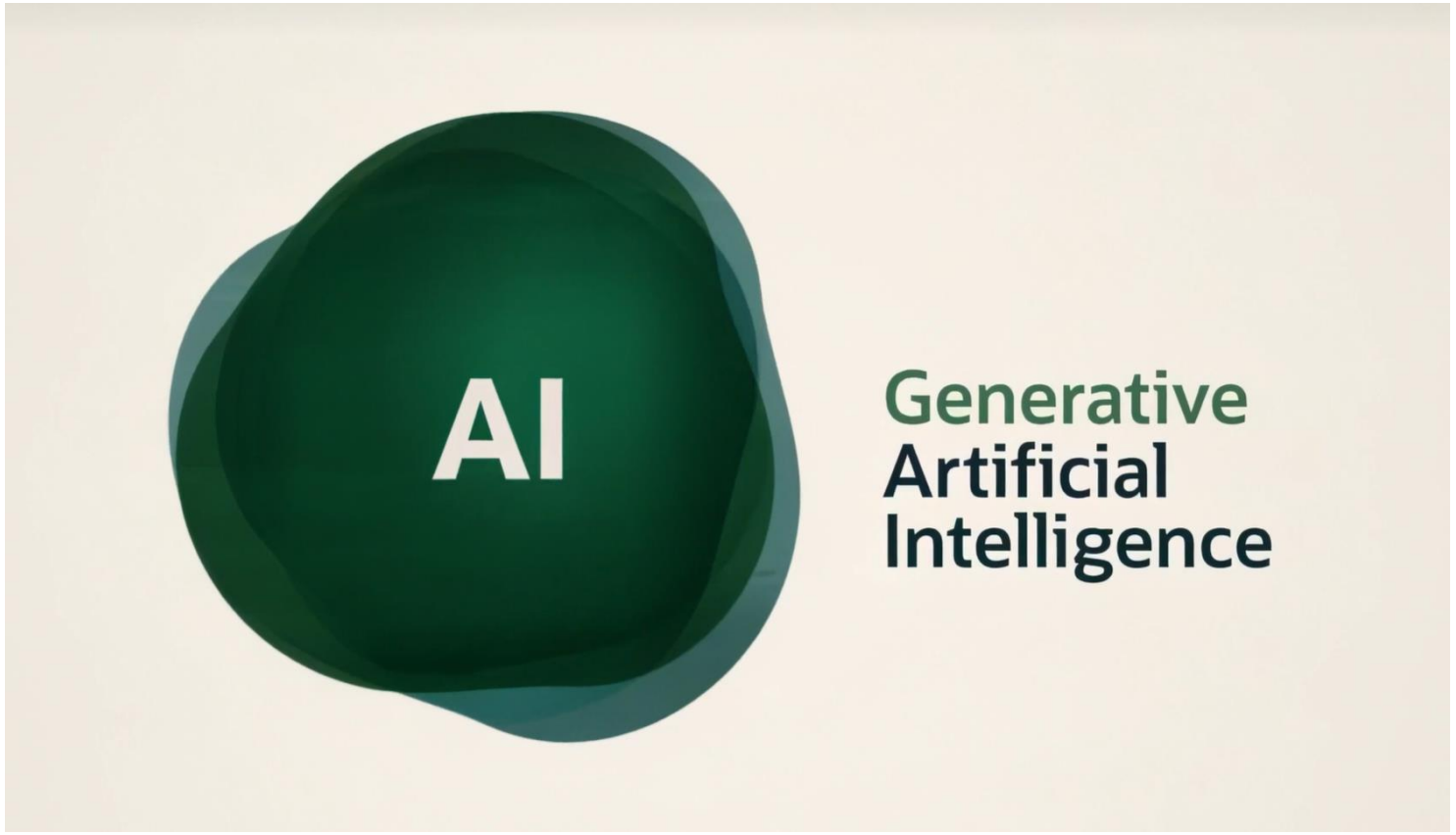
$$y_i = \gamma \hat{x}_i + \beta$$

  - 批量归一化添加在激活函数之前，解决训练速度慢、梯度爆炸等问题

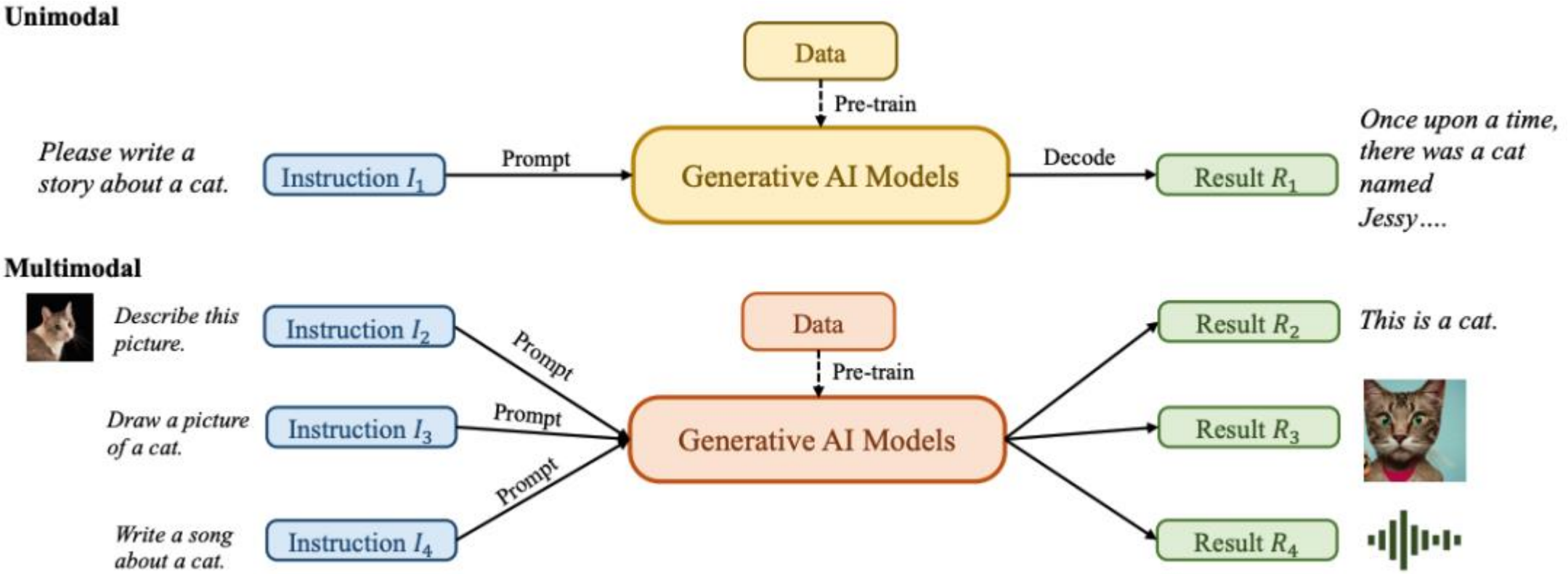
- 数据增强和辅助目标
  - 数据增强（data augmentation）
    - 扩增样本数量
    - 提高模型推广能力
  - 增加辅助目标
    - 增加提示性辅助目标改善方法的性能
    - 可以参与全部训练过程，也可以用作预训练

- 终止条件
  - 过学习提前停止
    - 划分一部分样本作为验证集
    - 训练误差继续减小但验证误差不变或反增
    - 此时可能出现过学习现象，应该提前停止训练
  - 网络结构或算法的问题
    - 训练误差和验证误差差别很大时就出现过学习现象
    - 此时可能网络结构和算法中存在问题，应该返回调整优化
  - “调参”需要观察神经网络训练全过程

12.10 生成式人工智能



生成式 AI（Generative Artificial Intelligence，GAI）是无监督和半监督的机器学习算法，其从数据中自我学习，并生成全新的、完全原始的数据，以保持与初始数据的相似性，可输出新的内容（包括文本、图像、视频和音频等）、合成数据和物理对象模型等。



一般来说，GAI模型可以分为两类：**单模态模型**和**多模态模型**，单模态模型从与生成的内容模态相同的模态接收指令，而多模态模型接受跨模态指令并产生不同模态的结果。

12.10.1 生成式人工智能的前世今生

（一）第一次浪潮（1956—1970年代）

1956年，达特茅斯学院人工智能夏季研讨会上正式提出使用人工智能（Artificial Intelligence, AI）这一术语。这是人类历史上第一次人工智能研讨，标志着人工智能学科的诞生。

1957年，佛兰克·罗森布拉特（Frank Rosenblatt）在一台IBM-704计算机上模拟实现了一种他发明的叫做“感知机”（Perceptron）的神经网络模型。这是最早的人工神经网络。

本阶段是让计算机具备逻辑推理能力。早期人工智能致力于解决计算机逻辑推理问题，并发明了解决代数和几何的程序，但是受硬件水平限制，计算机缺乏自学能力。这导致人工智能经历了近10年的低潮。

（二）第二次浪潮（1970—2006年）

1、应用发展（1980S—1970年代）

1979年，汉斯·贝利纳打造的计算机程序战胜双陆棋世界冠军成为标志性事件。

1982年，约翰·霍普菲尔德发明了霍普菲尔德网络，这是最早的RNN的雏形。

1989年，LeCun (CNN之父，Meta现任首席科学家) 发明了卷积神经网络，并首次将卷积神经网络成功应用到美国邮局的手写字符识别系统中。

阶段核心：**总结知识，并传递给计算机**。专家系统模拟人类专家的知识和经验解决特定领域问题，研究走向实际应用。

（二）第二次浪潮（1970—2006年）

2、平稳发展（1990s-2006年）

1997年，IBM深蓝技超级计算机战胜了国际象棋世界冠军卡斯帕罗夫，通过生成所有走法探索最佳棋局路径轰动全球。

2003年，Google公布了3篇大数据奠基性论文，为大数据存储及分布式处理的核心问题提供了思路：非结构化文件分布式存储（GFS）、分布式计算（MapReduce）及结构化数据存储（BigTable），并奠定了现代大数据技术的理论基础。

第一次算力和模型的爆发下，**人工智能研究的重心从知识系统转向了机器学习**，并推出数个关键的技术理论雏形。

（三）第三次浪潮（2006年至今）

2006年，杰弗里·辛顿以及他的学生正式提出了**深度学习**的概念（Deeping Learning）。

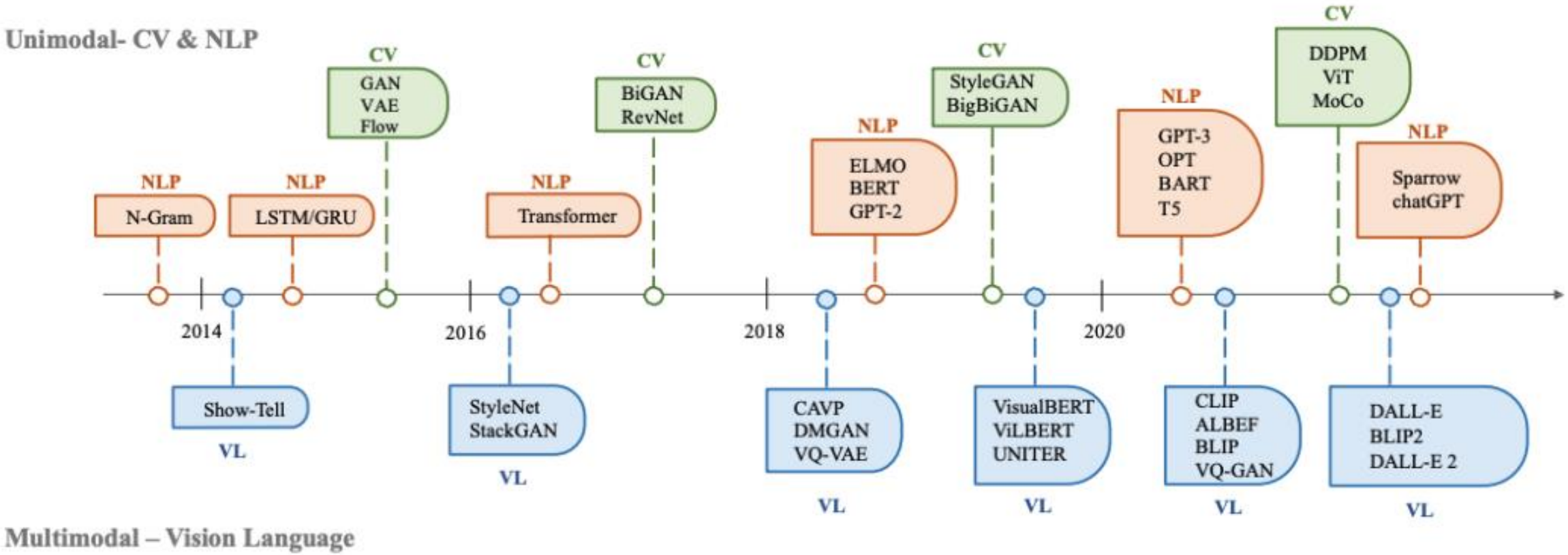
2006年，亚马逊AWS云计算平台发布，进一步大幅提升人工智能模型计算的算力。

2014年，4G时代的到来和智能手机大规模普及，带来了模型训练迭代所需的海量数据养分。

人工智能在诞生之初，社会对其充满了无限的乐观。然而，研究成果很快被证明缺乏实际作用，AI寒冬到来，人们的情绪反复在两端来回盘旋。

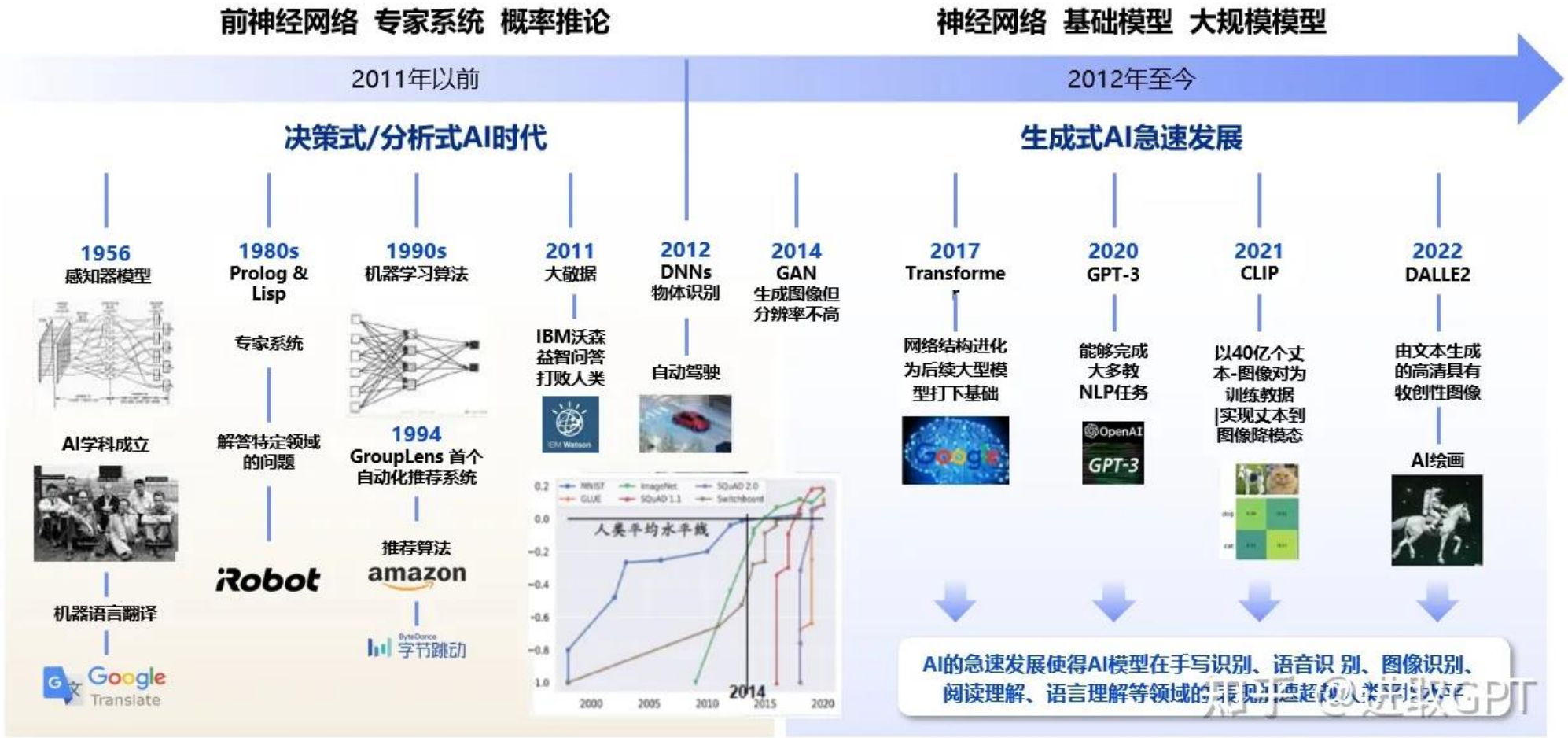
12.10.2 生成式AI在不同技术领域的发展

生成式AI在**CV（Computer Vision）**、**NLP（Natural Language Processing）**和**VL（Vision Language）**领域的发展历史。



12.10.3 生成式AI大规模商业化

随着**算法、算力和数据**三大要素的任意一项的迭代，人工智能发展也呈螺旋式上升，AI技术也得以逐步成熟和应用到各个领域。



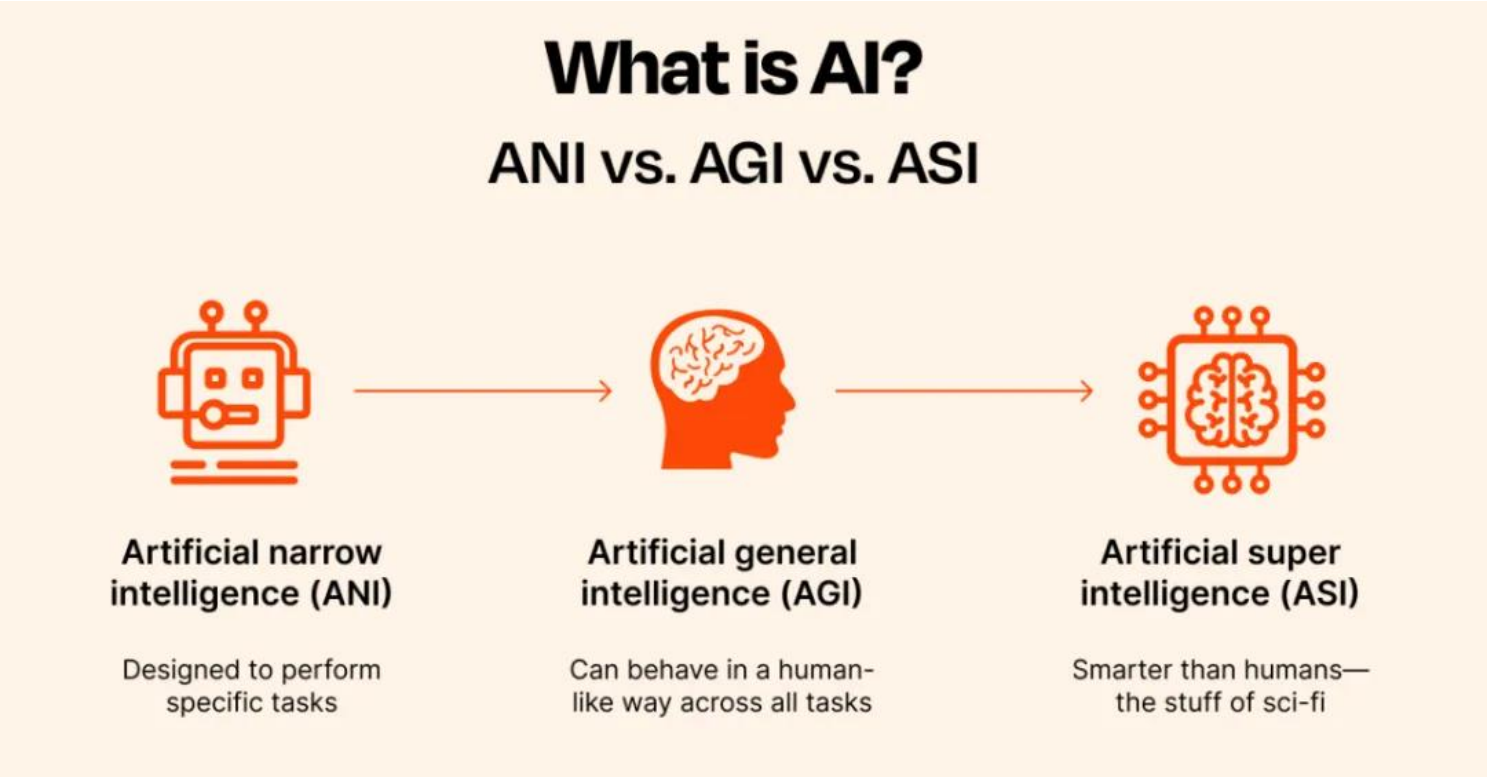
12.10.3 生成式AI大规模商业化

2016年开始，人工智能技术落地全面爆发，以计算机视觉为首，推荐算法、自然、语言处理等技术得到全面应用和发展，字节跳动、特斯拉等公司应运而起。



12.10.4 AIGC与GAI

AGI（人工通用智能）与**GAI（生成式人工智能）**不可混淆，AGI一直是科学家们世代追求的幻想，更不用说无数科幻电影和书籍了。值得注意的是，答案是“有条件的肯定”。在实际应用中，这些系统是多才多艺的“合成大脑”，但这并不意味着它们具有人类意义上的“思想”。它们没有独立的目标和欲望、偏见和愿望、情感和感觉：这些是独特的人类特征。但是，如果我们用正确的数据对它们进行训练并指导它们追求适当的目标，这些程序可以表现得好像具有这些特征一样。



12.11 人工智能范式的变化

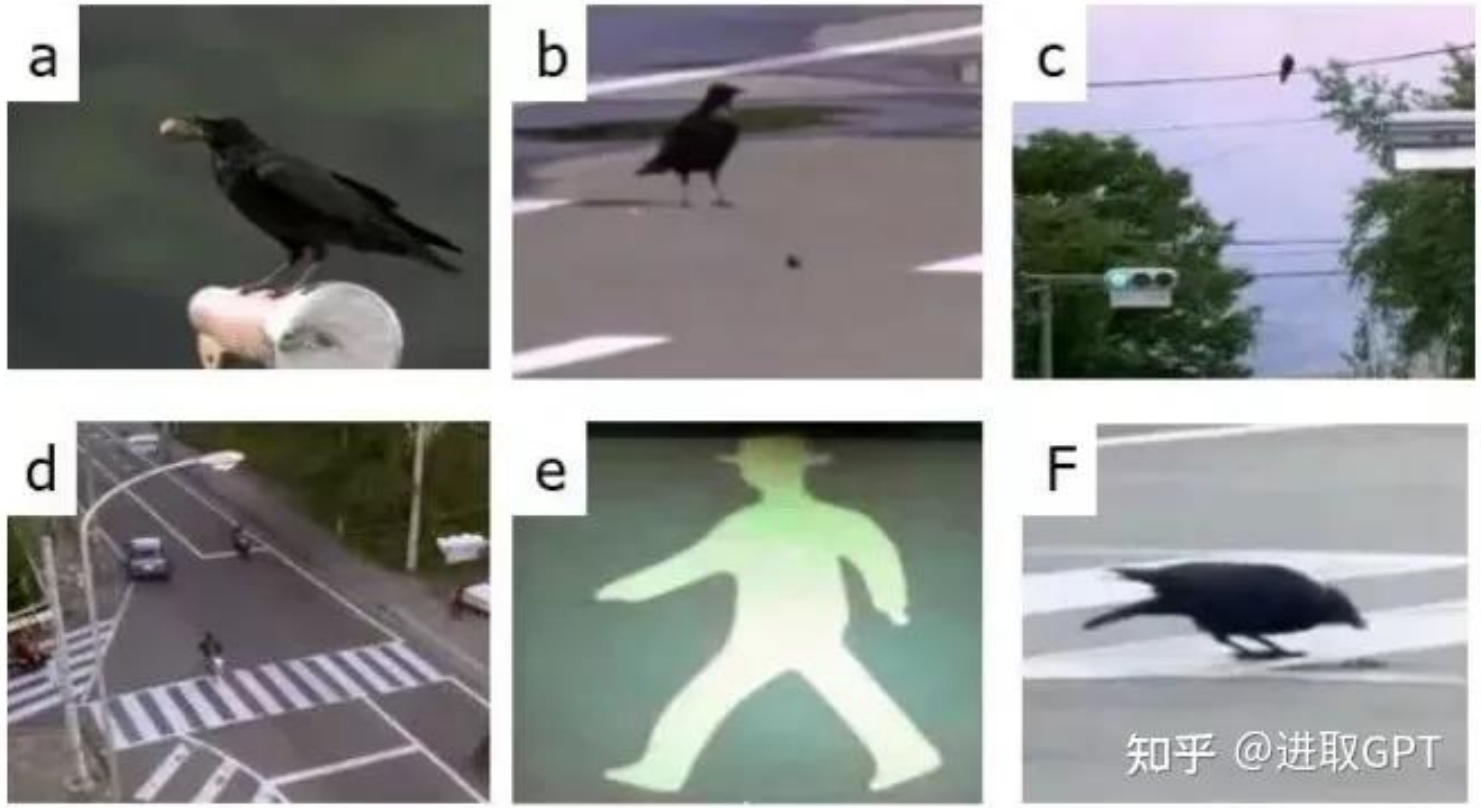
人工智能——过去的范式：

过去机器学习一直没有摆脱“Data Fitting”的基本范式，即找到数据的对应关系并应用。这有些像“鹦鹉学舌”，机器学习找到的对应关系仅仅能在已有信息包含的范畴之内总结规律。

Siri刚出来的时候，当时大家都曾觉得这是人工智能，但背后不过是NLP做的填表任务，语音助手从来没有真正“懂”这些指令。

人工智能—新的范式：

朱松纯教授在2017年有一篇思考人工智能和智能本质的文章，举了一个关于乌鸦和鹦鹉的例子，深刻揭示了什么是新的范式。



人工智能——过去的范式：

没有人教会乌鸦如何安全的吃到坚果，在情境中，乌鸦安全吃到坚果需要满足三个隐性条件：

1、掌握红绿灯（车）规律

2、车可以压碎坚果

3、车可以撞死乌鸦

过去的范式：鹦鹉学舌，所有乌鸦共用一个大脑，用穷举的方式尝试所有可能性，直到吃到坚果。

新的范式：尝试让自身学会乌鸦是如何“推理/推断”得到结论。

鹦鹉学舌：先听，再模仿，基于“对应关系”

乌鸦：先观察，再学习，基于“内在逻辑”

GPT-4看似拥有乌鸦的能力。

12.12 ChatGPT

GPT（Generative Pre-trained Transformer）生成式预训练

关键技术：Transformer、无监督预训练技术、微调技术、强化学习

GPT-o1（OpenAI于2024年9月发布）在博士级科学问题，超过了人类专家的平均水平





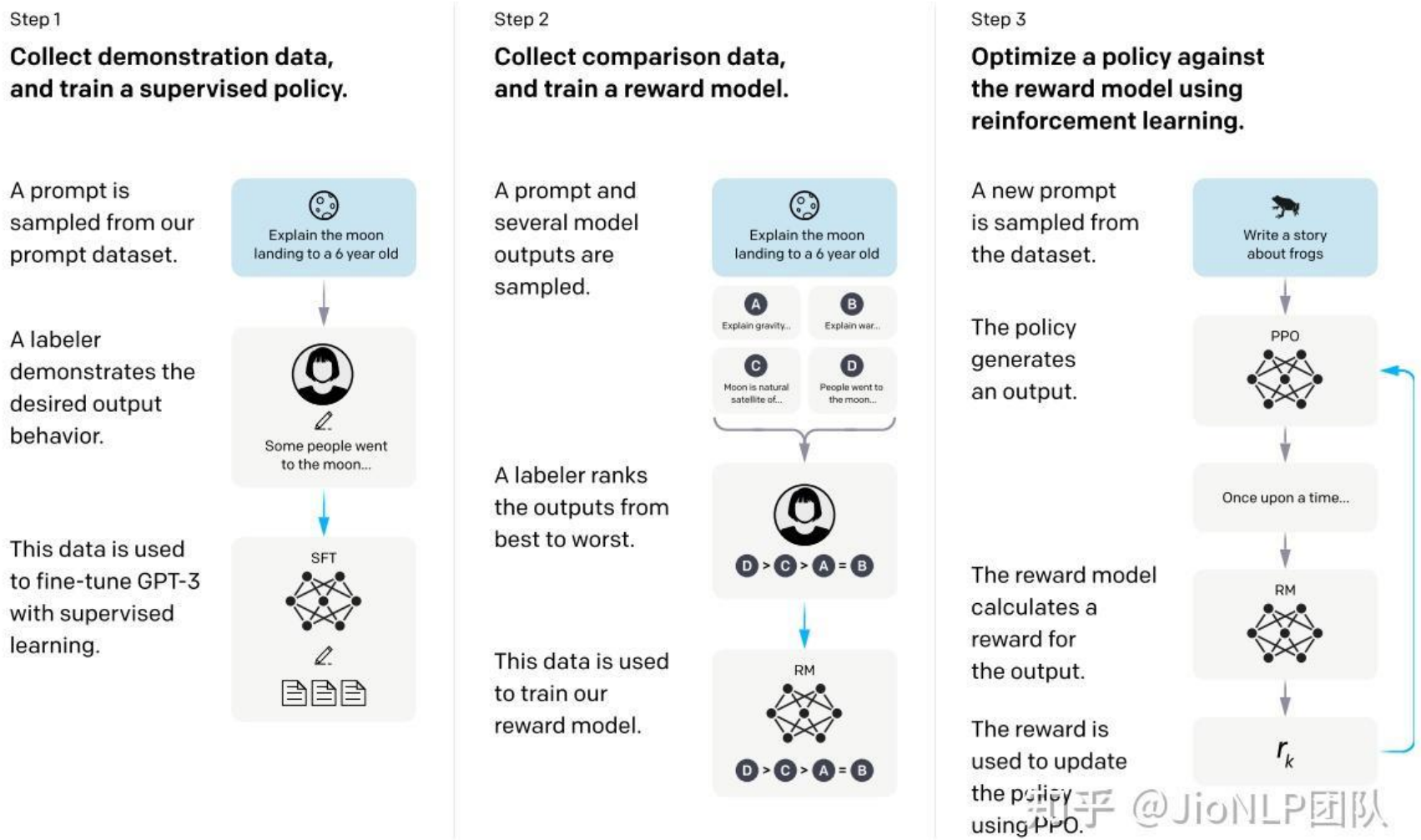






12.12.1 Chat GPT——训练过程

监督微调模型（SFT）、训练回报模型、使用 PPO 模型微调 SFT 模型

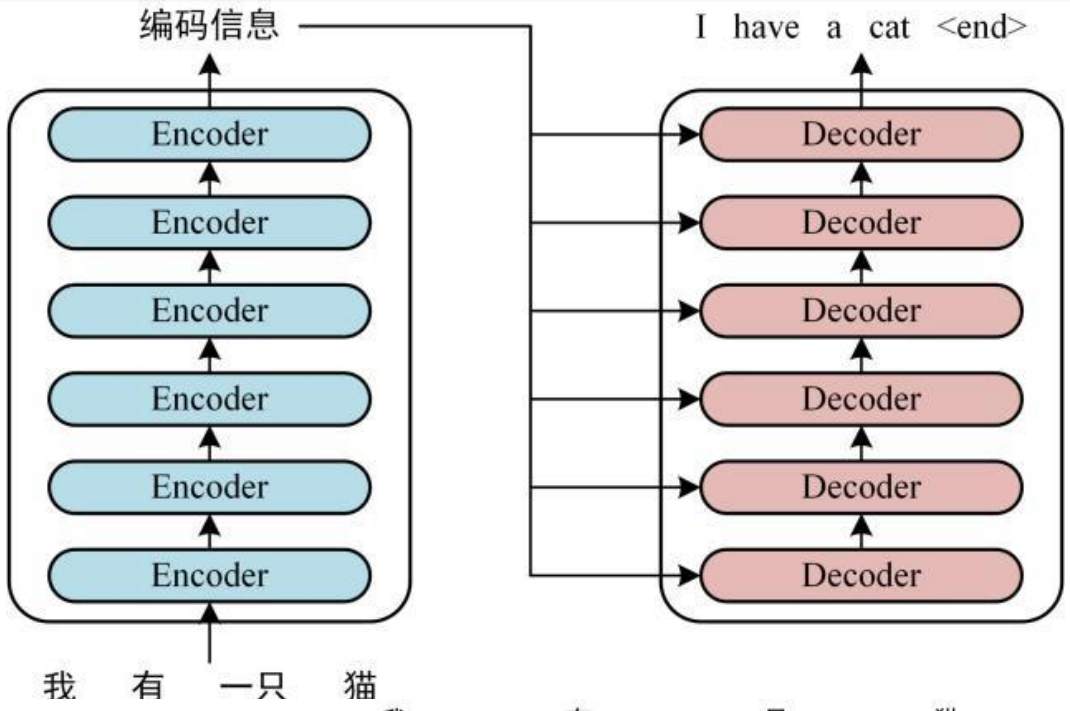


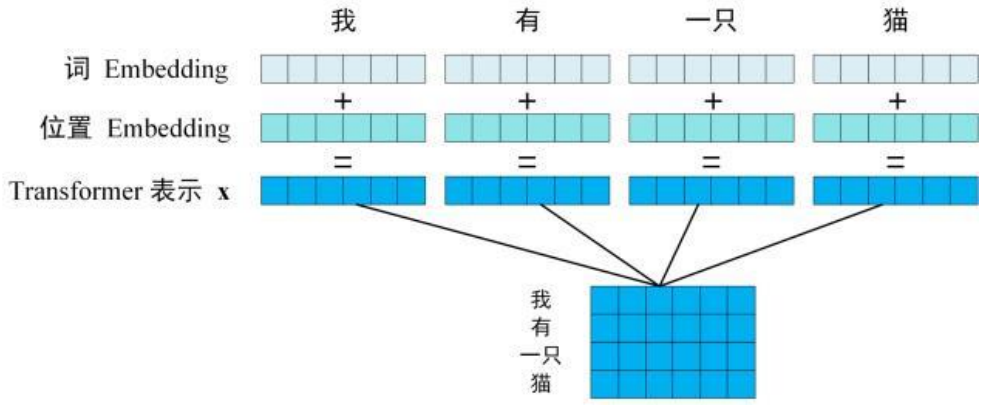
知乎 @JioNLP团队

12.12.2 Chat GPT——Transformer

论文：Vaswani A, Shazeer N, Parmar N, et al. Attention is all you need

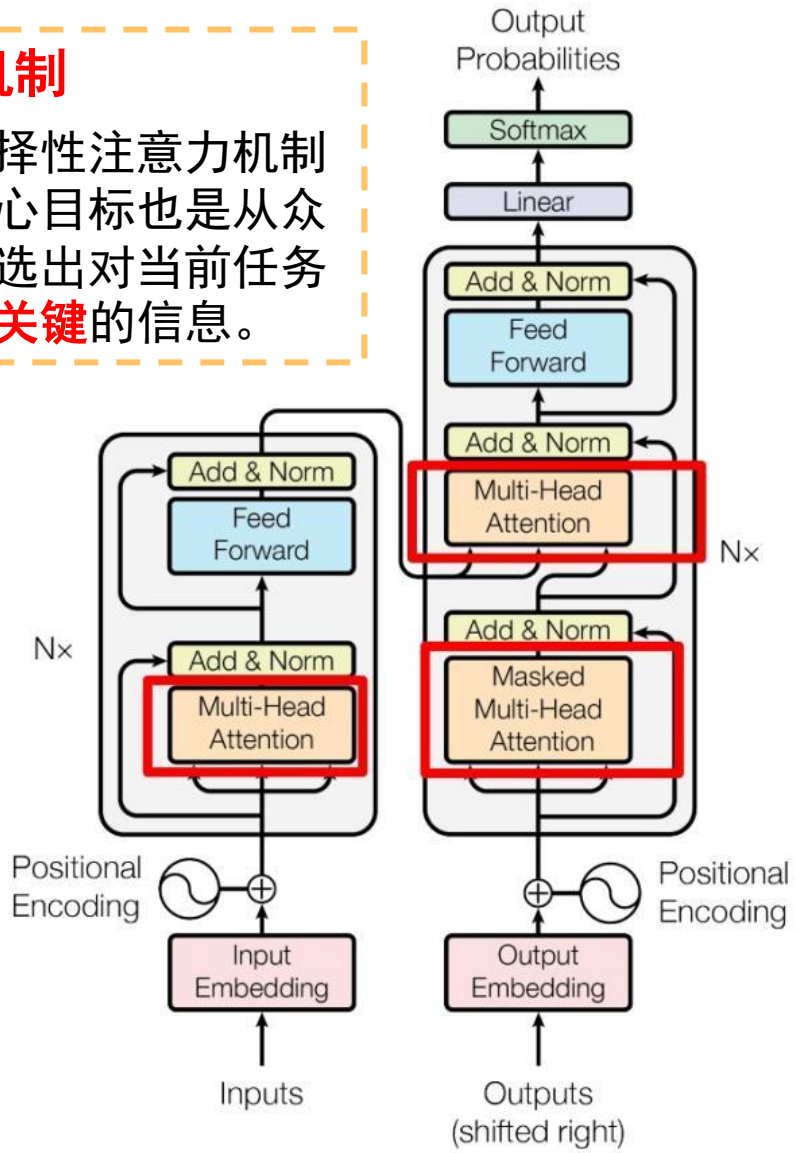
接受听到或读到的语言 — 大脑理解 — 输出要说的语言



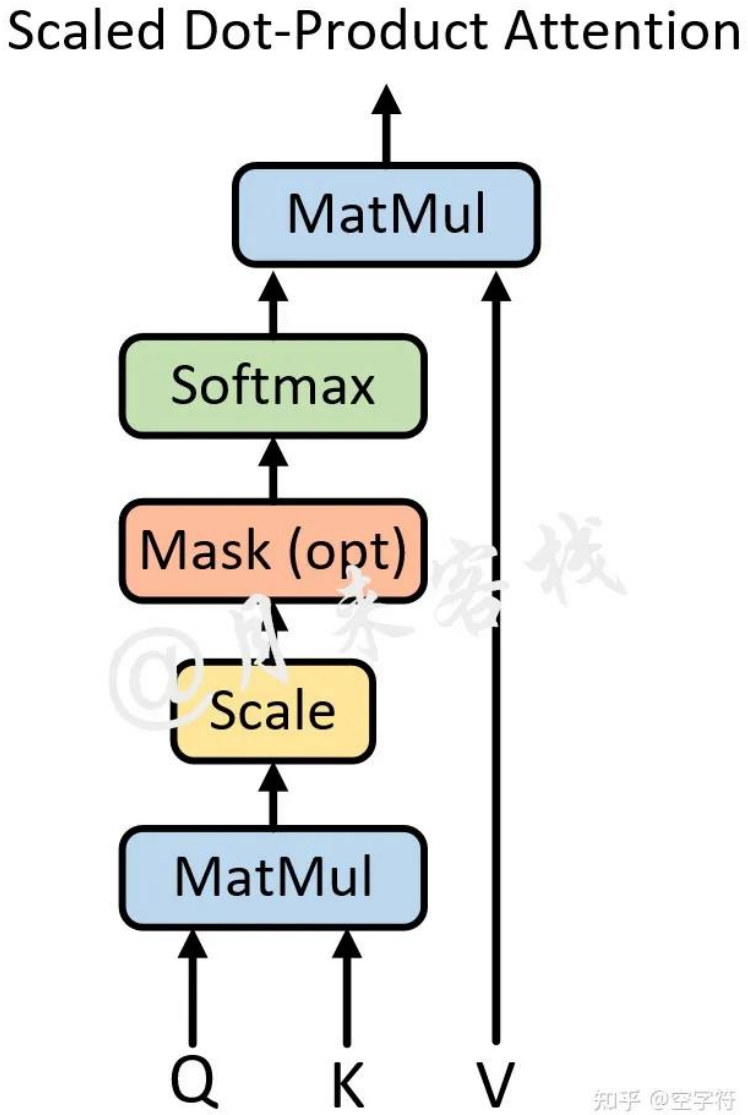


注意力机制

人类的选择性注意力机制类似，核心目标也是从众多信息中选出对当前任务目标更加**关键**的信息。



12.12.3 Chat GPT——自注意力机制



所谓的自注意力机制其实就是论文中所指代的“**Scaled Dot-Product Attention**“。在论文中作者说道，注意力机制可以描述为将query和一系列的key-value对映射到某个输出的过程，而这个输出的向量就是根据query和key计算得到的权重作用于value上的权重和。

$$\text{Attention}(Q,K,V)=\text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

其中**Q、K和V**分别为3个矩阵，且其（第2个）维度分别为$\mathrm{d}_q,\mathrm{d}_k,\mathrm{d}_v$（从后面的计算过程其实可以发现$\mathrm{d}_q=\mathrm{d}_v$。而上式中除以$\sqrt{\mathrm{d}_k}$的过程就是左图所指的**Scale**过程。

OpenAI连续12天AI发布会：

OpenAI于2024年12月5日启动了为期12天的特别活动，每个工作日通过直播发布一项全新的AI功能或产品。

Day 1 （12.5，太平洋时间UTC/GMT -8）：o1正式版和ChatGPT Pro

•发布了推理模型o1的正式版本，增强了编码、数学和写作能力，并支持图片上传等多模态功能。

•推出了每月订阅费200美元的ChatGPT Pro，提供无限制访问o1等高级功能

Day 2（12.6）：强化微调研究计划

•扩大了强化微调的研究计划，邀请研究人员、大学和企业申请参与

Day 3（12.9）：Sora

• 支持分辨率高达 1080p、时长最长 20 秒的视频，格式可选宽屏、竖屏或方形
• 提供文本转视频、图像转视频和视频转视频功能
• 配备精美的分镜工具，让您精准指导视频创作
• 包含“精选”和“最近”内容板块，从社区中汲取灵感
• 内置安全措施，确保透明性并防止滥用
• 通过 Plus 订阅可使用，或通过 Pro 订阅享受 10 倍使用量及更高分辨率

Day 4（12.10）：Canvas

- OpenAI 将面向所有用户开放 Canvas，并且直接将其整合到主模型中，免去了额外的加载步骤。
- 现在你可以在 Canvas 中运行 Python 代码，并实时查看文本或图形输出。
- 在用户定制的 GPT（Custom GPT）中也可以调用 Canvas，这样 GPT Store 上定制的 GPT 都能充分利用 Canvas 的强大功能。

Day 5（12.11）：ChatGPT集成至苹果系统

• ChatGPT现已集成到苹果的iOS、iPadOS和macOS系统，用户可直接在操作系统中访问ChatGPT的功能

Day 6（12.12）：高级语音与视频功能及圣诞模式

- 增强了ChatGPT的语音和视频功能，并推出了圣诞模式，增添节日氛围

Day 7（12.13）：ChatGPT中的项目功能

- ChatGPT 可基于项目内容提供动态建议，例如改进文本、优化代码或设计计划

12.13 讨论

- 深度学习在理论、方法和实现技术上都有很大的发展
- 需要避免“重调参，轻理论”的现象
- 需要进一步开展深度学习方法的理论研究